```
// ============================================================================
// prescription-core.js — 处方签核心共享模块
// 收缩到真实 API 面：全库触点审计（命名空间调用/动态访问/
// 工具链脚本）确认仅 getAutoJianfa 被消费（7 份 index.html 防御式调用）。
// 原 17 个工具函数（escapeHtml/formatPrice/排序/金额计算/校验/记录构建等）
// 自模块创建起即被各端 index.html 内联同名实现遮蔽、零命名空间调用，属
// 双轨漂移温床（如 escapeHtml 模块版 `!str` 守卫 vs 内联版 `str == null`
// 守卫已实际漂移），整体移除——唯一活实现 = 各端内联版，云↔离线同体由
// diff-cross-version.cjs Tier B 基线守护。
// 保留两个活特性：
// 1. PrescriptionCore.getAutoJianfa：药典煎煮方法自动匹配（JIANFA_RULES 唯一权威源）
// 2. PaperHistoryToggle：处方签「病史症状」栏开关（自执行运行时注入，HTML 零改动）
// ============================================================================
(function (global) {
    'use strict';
 // ==================== 中药煎煮方法规范规则表 ====================
 // 依据 2020 版《中国药典》及《中药学》教材特殊煎煮法归纳。
 // 优先级约定（各端调用）：药库手动设置(非普通煎) > 本规则表自动匹配 > 普通煎
    const JIANFA_RULES = {
 // 先煎 —— 矿物、贝壳、化石类及需久煎去毒的药物
        '石膏': '先煎', '寒水石': '先煎', '花蕊石': '先煎', '海浮石': '先煎',
        '海蛤壳': '先煎', '瓦楞子': '先煎', '龙骨': '先煎', '龙齿': '先煎',
        '牡蛎': '先煎', '石决明': '先煎', '珍珠母': '先煎', '代赭石': '先煎',
        '磁石': '先煎', '紫石英': '先煎', '自然铜': '先煎', '龟甲': '先煎',
        '鳖甲': '先煎', '水牛角': '先煎', '川乌': '先煎', '草乌': '先煎', '附子': '先煎',
 // 后下 —— 气味芳香含挥发油、久煎降效的药物
        '薄荷': '后下', '砂仁': '后下', '白豆蔻': '后下', '青蒿': '后下',
        '香薷': '后下', '钩藤': '后下', '大黄': '后下', '番泻叶': '后下',
        '沉香': '后下', '檀香': '后下', '降香': '后下', '鱼腥草': '后下',
 // 包煎 —— 细小种子、花粉、含绒毛易浑汤的药物
        '车前子': '包煎', '滑石': '包煎', '旋覆花': '包煎', '辛夷': '包煎',
        '枇杷叶': '包煎', '海金沙': '包煎', '蒲黄': '包煎', '葶苈子': '包煎', '蚕沙': '包煎',
 // 烊化 —— 胶类药（另溶化兑入）
        '阿胶': '烊化', '鹿角胶': '烊化', '龟甲胶': '烊化', '鳖甲胶': '烊化',
        '黄明胶': '烊化', '饴糖': '烊化',
 // 另煎 —— 贵细药材（单独煎煮兑入）
        '人参': '另煎', '红参': '另煎', '西洋参': '另煎', '羚羊角': '另煎',
        '鹿茸': '另煎', '西红花': '另煎',
 // 冲服 —— 粉末/树脂类宜研末冲服（不入煎剂）
        '三七': '冲服', '琥珀': '冲服', '珍珠': '冲服', '牛黄': '冲服',
        '麝香': '冲服', '冰片': '冲服', '芒硝': '冲服', '玄明粉': '冲服',
        '竹沥': '冲服', '雷丸': '冲服',
 // 煎汤代水 —— 先单独煎煮取汤，再以其汤代水煎其余药物
        '灶心土': '煎汤代水', '伏龙肝': '煎汤代水',
 // 兑服 —— 液汁类直接兑入药液
        '姜汁': '兑服'
    };
 // 名称包含误匹配排除表（如"香附子"含"附子"、"肉豆蔻"含"豆蔻"，不得命中先煎/后下）
    const JIANFA_EXCLUDE = ['香附', '肉豆蔻'];
 // 按药名自动匹配规范煎煮方法：精确命中 > 长关键词包含匹配 > 无（返回''）
 // 支持"煅龙骨/生石膏/炒车前子"等炮制前缀写法
    function getAutoJianfa(name) {
```

```
        if (!name) return '';
        const n = String(name).replace(/\s+/g, '');
        if (JIANFA_RULES[n]) return JIANFA_RULES[n];
        for (let i = 0; i < JIANFA_EXCLUDE.length; i++) {
            if (n.indexOf(JIANFA_EXCLUDE[i]) !== -1) return '';
        }
        const keys = Object.keys(JIANFA_RULES).sort((a, b) => b.length - a.length);
        for (let i = 0; i < keys.length; i++) {
            if (n.indexOf(keys[i]) !== -1) return JIANFA_RULES[keys[i]];
        }
        return '';
    }
 // ==================== 导出 ====================
    const PrescriptionCore = {
        getAutoJianfa
    };
    global.PrescriptionCore = PrescriptionCore;
})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
// ================================================================================
// 处方签「病史症状」栏显示/隐藏开关（ · 运行时注入 · HTML 零改动）
// 需求：处方签病史症状栏可勾选显示/隐藏（localStorage 持久化）；
// 默认隐藏 —— 屏幕预览与打印一致（打印取 #prescriptionPaper.innerHTML，
// 本模块对行内 div 写 style.display:none，随 innerHTML 携带进打印窗口）
// ================================================================================
(function (global) {
    'use strict';
    if (global.__paperHistoryToggleLoaded) return;
    global.__paperHistoryToggleLoaded = true;
    var LS_KEY = 'local_paperShowHistory'; // '1'=显示 / 其他=隐藏（默认隐藏）
    function getShow() {
        try { return !!(global.localStorage && global.localStorage.getItem(LS_KEY) === '1'); }
        catch (e) { return false; }
    }
    function setShow(v) {
        try { if (global.localStorage) global.localStorage.setItem(LS_KEY, v ? '1' : '0'); } catch (e) {}
    }
 // 处方签上「病史症状」整行 = #paperMedicalHistory 向上最近的块级 div
 //（该 div 内含文本 span + 下划分隔线，整行隐藏即为栏目隐藏）
    function findRow() {
        var s = document.getElementById('paperMedicalHistory');
        if (!s) return null;
        if (s.closest) return s.closest('div');
        return s.parentElement && s.parentElement.parentElement;
    }
 // 应用显示/隐藏到处方签行 + 同步勾选框状态
    function applyHistoryToggle() {
        var row = findRow();
        if (row) row.style.display = getShow() ? '' : 'none';
        var cb = document.getElementById('paperHistoryToggle');
        if (cb && cb.checked !== getShow()) cb.checked = getShow();
    }
 // 打印前兜底：包一层 printPrescription，进打印前再 apply 一次
```

```
//（防其他逻辑复位 display：printPrescription 由各端内联脚本后定义，需延迟包裹）
    function wrapPrint() {
        if (typeof global.printPrescription === 'function' && !global.printPrescription.__histWrapped) {
            var orig = global.printPrescription;
            var wrapped = function () { applyHistoryToggle(); return orig.apply(this, arguments); };
            wrapped.__histWrapped = true;
            global.printPrescription = wrapped;
            return true;
        }
        return typeof global.printPrescription === 'function';
    }
// 注入勾选框到左栏「病史症状」页签行右端（不在 #prescriptionPaper 内，永不进打印）
    function injectHistoryToggle(retry) {
        retry = retry || 0;
        var row = findRow();
        var tabs = document.querySelector('.symptom-section .history-tabs');
        if (!row || !tabs) {
            if (retry < 40) setTimeout(function () { injectHistoryToggle(retry + 1); }, 250);
            return;
        }
        if (!document.getElementById('paperHistoryToggle')) {
            var lab = document.createElement('label');
            lab.id = 'paperHistoryToggleWrap';
            lab.style.cssText = 'margin-left:auto;display:flex;align-items:center;gap:4px;font-size:11px;color:#555;
cursor:pointer;white-space:nowrap;padding:2px 0 2px 8px;';
            lab.title = '勾选=处方签显示病史症状栏（屏幕预览与打印一致）；默认隐藏、打印不输出';
            var cb = document.createElement('input');
            cb.type = 'checkbox';
            cb.id = 'paperHistoryToggle';
            cb.style.cssText = 'width:13px;height:13px;margin:0;cursor:pointer;accent-color:#006400;';
            var tx = document.createElement('span');
            tx.textContent = '处方签显示病史';
            lab.appendChild(cb);
            lab.appendChild(tx);
            tabs.appendChild(lab);
            cb.addEventListener('change', function () { setShow(cb.checked); applyHistoryToggle(); });
        }
        applyHistoryToggle();
    }
// 启动：勾选框/应用立即执行；printPrescription 包裹在后台重试
//（printPrescription 由各端内联脚本在页面加载期定义，本模块先加载）
    function bootHistoryToggle() {
        injectHistoryToggle(0);
        (function wrapRetry(retry) {
            retry = retry || 0;
            if (wrapPrint() || retry >= 40) return;
            setTimeout(function () { wrapRetry(retry + 1); }, 250);
        })(0);
    }
    if (document.readyState === 'loading') {
        document.addEventListener('DOMContentLoaded', bootHistoryToggle);
    } else {
```

```
        bootHistoryToggle();
    }
    global.PaperHistoryToggle = {
        isShown: getShow,
        setShown: function (v) { setShow(!!v); applyHistoryToggle(); },
        apply: applyHistoryToggle
    };
})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
// ================================================================================
// 处方签「金额」显示/隐藏开关（ · 运行时注入 · HTML 零改动）
// 需求：处方笺底部「金额:」栏是否显示实际金额（总计）可勾选（localStorage 持久化）；
// 默认隐藏（金额:留空供手写）；勾选后显示「金额:<总计>元」。
// 勾选框注入到操作区价格行（总计:xxx 元 右侧空位），不在 #prescriptionPaper 内，
// 永不进打印；打印取 #prescriptionPaper.innerHTML，金额文本随 innerHTML 携带。
// ================================================================================
(function (global) {
    'use strict';
    if (global.__paperAmountToggleLoaded) return;
    global.__paperAmountToggleLoaded = true;
    var LS_KEY = 'local_paperShowAmount'; // '1'=显示金额 / 其他=隐藏（默认隐藏）
    function getShow() {
        try { return !!(global.localStorage && global.localStorage.getItem(LS_KEY) === '1'); }
        catch (e) { return false; }
    }
    function setShow(v) {
        try { if (global.localStorage) global.localStorage.setItem(LS_KEY, v ? '1' : '0'); } catch (e) {}
    }
 // 处方笺底部 «金额:» 项 = prescription-footer 内 signature-item（文本以「金额」开头）
    function findAmountEl() {
        var paper = document.getElementById('prescriptionPaper');
        if (!paper) return null;
        var items = paper.querySelectorAll('.prescription-footer .signature-item');
        for (var i = 0; i < items.length; i++) {
            var txt = (items[i].textContent || '').replace(/\s+/g, '');
            if (txt.indexOf('金额') === 0) return items[i];
        }
        return null;
    }
 // 读取操作区「总计」当前金额（纯数字，去掉「元」等符号）
    function getGrandTotal() {
        var el = document.getElementById('grandTotal2') || document.getElementById('grandTotal');
        if (!el) return '';
        return (el.textContent || '').replace(/[^\d.\-]/g, '').trim();
    }
 // 应用显示/隐藏到处方笺金额栏 + 同步勾选框状态
    function applyAmountToggle() {
        var el = findAmountEl();
        if (!el) return;
        var cb = document.getElementById('paperAmountToggle');
        if (cb && cb.checked !== getShow()) cb.checked = getShow();
        if (getShow()) {
            var v = getGrandTotal();
```

```
            el.textContent = '金额:' + (v ? v + '元' : '');
        } else {
            el.textContent = '金额:';
        }
    }
// 打印前兜底：包一层 printPrescription，进打印前再 apply 一次
    function wrapPrint() {
        if (typeof global.printPrescription === 'function' && !global.printPrescription.__amountWrapped) {
            var orig = global.printPrescription;
            var wrapped = function () { applyAmountToggle(); return orig.apply(this, arguments); };
            wrapped.__amountWrapped = true;
            global.printPrescription = wrapped;
            return true;
        }
        return typeof global.printPrescription === 'function';
    }
// 注入勾选框到操作区价格行（「总计:xxx 元」右侧空位；不在 #prescriptionPaper 内，永不进打印）
    function injectAmountToggle(retry) {
        retry = retry || 0;
        var row = document.querySelector('.action-buttons-section > div');
        if (!row) {
            if (retry < 40) setTimeout(function () { injectAmountToggle(retry + 1); }, 250);
            return;
        }
        if (!document.getElementById('paperAmountToggle')) {
            var lab = document.createElement('label');
            lab.id = 'paperAmountToggleWrap';
            lab.style.cssText = 'margin-left:auto;display:flex;align-items:center;gap:4px;font-size:11px;color:#555;
cursor:pointer;white-space:nowrap;padding:2px 0 2px 8px;flex:0 0 auto;';
            lab.title = '勾选=处方签显示金额（金额:总计元）；默认隐藏、打印不输出金额';
            var cb = document.createElement('input');
            cb.type = 'checkbox';
            cb.id = 'paperAmountToggle';
            cb.style.cssText = 'width:13px;height:13px;margin:0;cursor:pointer;accent-color:#006400;';
            var tx = document.createElement('span');
            tx.textContent = '处方签显示金额';
            lab.appendChild(cb);
            lab.appendChild(tx);
            row.appendChild(lab);
            cb.addEventListener('change', function () { setShow(cb.checked); applyAmountToggle(); });
        }
        applyAmountToggle();
    }
// 监听「总计」数值变化，实时同步处方笺金额（勾选状态下）
    function watchGrandTotal() {
        var el = document.getElementById('grandTotal2') || document.getElementById('grandTotal');
        if (!el) { setTimeout(watchGrandTotal, 250); return; }
        if (el.__amountObserver) return;
        try {
            var mo = new MutationObserver(function () { if (getShow()) applyAmountToggle(); });
            mo.observe(el, { childList: true, characterData: true, subtree: true });
            el.__amountObserver = mo;
```

```
        } catch (e) {}
    }
    function bootAmountToggle() {
        injectAmountToggle(0);
        watchGrandTotal();
        (function wrapRetry(retry) {
            retry = retry || 0;
            if (wrapPrint() || retry >= 40) return;
            setTimeout(function () { wrapRetry(retry + 1); }, 250);
        })(0);
    }
    if (document.readyState === 'loading') {
        document.addEventListener('DOMContentLoaded', bootAmountToggle);
    } else {
        bootAmountToggle();
    }
    global.PaperAmountToggle = {
        isShown: getShow,
        setShown: function (v) { setShow(!!v); applyAmountToggle(); },
        apply: applyAmountToggle
    };
})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
// ================================================================================
// 处方笺「用法/剂数」行贴底横线（ · 运行时注入 · HTML 零改动）
// 需求：处方网格内「剂数:…水煎服…」用法行紧贴底部横线上方——药品少时不再悬在
// 网格中间、与底部横线之间留一大段空白。
// 做法：处方网格改纵向 flex，药品行容器 #prescriptionLines 占满剩余空间（flex:1），
// 用法行底距归零，自然被推到底部横线正上方。
// 仅写内联样式（随 #prescriptionPaper.innerHTML 带入打印窗口，屏幕/打印一致）。
// ================================================================================
(function (global) {
    'use strict';
    if (global.__paperUsageLayoutLoaded) return;
    global.__paperUsageLayoutLoaded = true;
    function applyUsageLayout() {
        var grid = document.querySelector('.prescription-grid');
        if (!grid) return false; // 处方网格未就绪，稍后重试
        var usage = grid.querySelector('.usage-text');
        if (!usage) return false; // 用法行未就绪，稍后重试
        grid.style.display = 'flex';
        grid.style.flexDirection = 'column';
        var lines = grid.querySelector('#prescriptionLines');
        if (lines) { lines.style.flex = '1 1 auto'; lines.style.minHeight = '0'; }
        usage.style.marginBottom = '0';
        return true;
    }
    function bootUsageLayout() {
        (function retry(n) {
            if (applyUsageLayout()) return;
            if (n >= 40) return;
            setTimeout(function () { retry(n + 1); }, 250);
        })(0);
```

```
    }
    if (document.readyState === 'loading') {
        document.addEventListener('DOMContentLoaded', bootUsageLayout);
    } else {
        bootUsageLayout();
    }
})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
// ============================================================================
// 处方笺「中药末位行 → 用法行」斜线封口（ · 运行时注入 · HTML 零改动）
// 需求：B5 纸面下药品末位行与「用法：」行之间常留一段空白，用左下→右上斜线
// 封口（参考手写处方笺习惯），既填补空白也防止空白区被误加药。
// 做法：#prescriptionLines 改纵向 flex，末尾放一个 flex:1 的占位块，块内内联 SVG
// 斜线按实际剩余高度自适应（preserveAspectRatio=none / 非缩放描边）。
// 全部写内联样式，随 #prescriptionPaper.innerHTML 同步进打印窗口，屏幕/打印一致。
// ============================================================================
(function (global) {
    'use strict';
    if (global.__paperEndSlashLoaded) return;
    global.__paperEndSlashLoaded = true;
    var SLASH_ID = 'rxEndSlash';
    var SLASH_SETTING_KEY = 'local_rxEndSlashVisible';
    var SLASH_SVG = '<svg xmlns="http://www.w3.org/2000/svg" width="100%" height="100%" viewBox="0 0 100 100"' +
        ' preserveAspectRatio="none" aria-hidden="true" style="display:block;width:100%;height:100%;">' +
        '<line x1="0" y1="100" x2="100" y2="0" stroke="#000" stroke-width="1.2"' +
        ' vector-effect="non-scaling-stroke" stroke-linecap="butt"></line></svg>';
 // 斜线显示开关（默认开）：F1 基础设置里勾选/取消，值存 localStorage，多端各自记忆
    function slashEnabled() {
        try {
            return global.localStorage.getItem(SLASH_SETTING_KEY) !== '0';
        } catch (e) {
            return true;
        }
    }
    global.setRxEndSlashEnabled = function (on) {
        try { global.localStorage.setItem(SLASH_SETTING_KEY, on ? '1' : '0'); } catch (e) {}
        applyEndSlash();
    };
    function ensureSlash(lines) {
        var filler = document.getElementById(SLASH_ID);
        if (!filler) {
            filler = document.createElement('div');
            filler.id = SLASH_ID;
            filler.setAttribute('aria-hidden', 'true');
 // line-height/overflow 归零：斜线块只占剩余空白，不额外撑高纸面
            filler.style.cssText = 'flex:1 1 auto;min-height:0;align-self:stretch;pointer-events:none;line-height:0;
overflow:hidden;';
            filler.innerHTML = SLASH_SVG;
        }
 // updatePrescriptionPaper 会整段重写 #prescriptionLines.innerHTML，每次重新压到末位
        if (lines.lastElementChild !== filler) lines.appendChild(filler);
        return filler;
    }
```

```
    function applyEndSlash() {
        var grid = document.querySelector('.prescription-grid');
        if (!grid) return false; // 处方网格未就绪，稍后重试
        var lines = grid.querySelector('#prescriptionLines');
        var paper = document.getElementById('prescriptionPaper');
// 用法行位置有两种版式：云/离线正文用法行在网格内（site-admin 在页脚内）→ 兜底全局查找
        var usage = grid.querySelector('.usage-text') || (paper ? paper.querySelector('.usage-text') : null);
        if (!lines || !usage) return false; // 药品行/用法行未就绪
        var inner = lines.querySelector('.prescription-grid-inner');
// 手机端预览已改为 A5 固定纸 + 整页等比缩放（与桌面同一套纸面），
// 固定纸高下的「剩余空白」同样存在，故不再按视口宽度跳过斜线，手机与桌面一致。
// 纸面纵向 flex：表头(诊所名/标题/患者信息/病史/诊断)高度按内容固定，
// 网格占满剩余空间 → 结尾区(用法行+签名栏)被压到纸面底部，位置不随药味数浮动。
        if (paper) {
            for (var i = 0; i < paper.children.length; i++) {
                var ch = paper.children[i];
                if (ch !== grid) ch.style.flex = '0 0 auto';
            }
        }
        grid.style.flex = '1 1 auto';
        grid.style.minHeight = '0'; // 覆盖 .prescription-grid 的 min-height，药味多时允许压缩
        grid.style.display = 'flex';
        grid.style.flexDirection = 'column';
// 网格内：RP 行 / 用法行固定高度，药品行容器占满中段
        var rp = grid.querySelector('.prescription-line');
        if (rp) rp.style.flex = '0 0 auto';
// 用法行仅在网格内（云/离线版式）时参与网格 flex；site-admin 版式在页脚内，跳过
        if (usage.parentNode === grid) {
            usage.style.flex = '0 0 auto';
            usage.style.marginBottom = '0';
        }
        lines.style.display = 'flex';
        lines.style.flexDirection = 'column';
        lines.style.flex = '1 1 auto';
        lines.style.minHeight = '0';
        if (inner) inner.style.flex = '0 0 auto'; // 药品行按内容高度，不参与拉伸
// 斜线块 flex:1：高度 = 纸面剩余高度 − 已用药味行高度，随药味行数自动伸缩
        var filler = ensureSlash(lines);
// 无药味时不出斜线（空网格整页斜线无意义）；F1 设置未勾选时也不显示
        var rowCount = inner ? inner.querySelectorAll('.prescription-line').length : 0;
        filler.style.display = (rowCount > 0 && slashEnabled()) ? 'block' : 'none';
        return true;
    }
// F1 基础设置里注入「处方笺斜线」勾选框（运行时注入，不改 HTML 静态结构）
    function injectSettingsToggle() {
        var modal = document.getElementById('settingsModal');
        if (!modal) return false;
        if (document.getElementById('rxEndSlashSettingRow')) return true;
        var body = modal.querySelector('.modal-body');
        if (!body) return false;
        var row = document.createElement('div');
        row.id = 'rxEndSlashSettingRow';
```

```
        row.style.cssText = 'margin-bottom:12px;';
        var label = document.createElement('label');
        label.style.cssText = 'display:flex;align-items:center;gap:8px;cursor:pointer;';
        var cb = document.createElement('input');
        cb.type = 'checkbox';
        cb.id = 'rxEndSlashToggle';
        cb.style.cssText = 'width:18px;height:18px;';
        cb.checked = slashEnabled();
        cb.addEventListener('change', function () { global.setRxEndSlashEnabled(cb.checked); });
        var txt = document.createElement('span');
        txt.textContent = '处方笺显示中药末行斜线';
        label.appendChild(cb);
        label.appendChild(txt);
        var hint = document.createElement('div');
        hint.style.cssText = 'font-size:11px;color:#666;margin-top:4px;';
        hint.textContent = '勾选后，中药末位行到「用法」之间自动画斜线，防止空白处被误加药；取消勾选即不显示。';
        row.appendChild(label);
        row.appendChild(hint);
        body.appendChild(row);
        return true;
    }
 // 监听药品渲染重写，实时补回斜线块（否则每次输入都会被 innerHTML 清掉）
    function watchLines() {
        var lines = document.querySelector('#prescriptionLines');
        if (!lines) return false;
        if (lines.__rxEndSlashObserver) return true;
        try {
            var mo = new MutationObserver(function () { applyEndSlash(); });
            mo.observe(lines, { childList: true });
            lines.__rxEndSlashObserver = mo;
            return true;
        } catch (e) {
            return false;
        }
    }
 // 打印前兜底：确保 updatePrescriptionPaper 与 printPrescription 同一任务内执行时斜线也已就位
    function wrapPrint() {
        if (typeof global.printPrescription === 'function' && !global.printPrescription.__endSlashWrapped) {
            var orig = global.printPrescription;
            var wrapped = function () { applyEndSlash(); return orig.apply(this, arguments); };
            wrapped.__endSlashWrapped = true;
            global.printPrescription = wrapped;
            return true;
        }
        return typeof global.printPrescription === 'function';
    }
 // 打开基础设置时同步勾选状态（多标签页/其他入口改过 localStorage 也不跑偏）
    function syncToggleState() {
        var cb = document.getElementById('rxEndSlashToggle');
        if (cb) cb.checked = slashEnabled();
/* ========== 源文件：index.html（处方编号唯一收口：就绪闸 / 三级持久化 / 权威集扫描） ========== */
 // 处方号生成唯一收口点（generatePrescriptionNo /
```

```
// generatePrescriptionNoForSave / 打印 / 图片 / 复制 / 新处方 共 7 处调用点都经它）。
// 未就绪（loadData 尚未把处方全量集放好）时 fail-closed：
// · 不读序列号、不写任何存储（绝不留下半成品序号让后续"接着往后取"）；
// · 返回 null 并给明确中文提示 —— 医疗场景"拒绝一次开方"远优于"生成重复处方号"。
// 就绪后行为与改动前逐字一致（当日已存序号直接返回；否则扫权威处方集取当日最大）。
       async function getGlobalMaxSerial() {
// ① 就绪闸（带超时；超时同样 fail-closed）
            const __readyFlag = (typeof window !== 'undefined') && window.__prescriptionsReady === true;
            const __readyPromise = (typeof window !== 'undefined') ? window.__prescriptionsReadyPromise : null;
            if (!__readyFlag && __readyPromise && typeof __readyPromise.then === 'function') {
                let __timeoutMs = 10000;
                try {
                    const __ov = (typeof window !== 'undefined') ? Number(window.__prescriptionsReadyTimeoutMs) :
NaN;
                    if (isFinite(__ov) && __ov > 0) __timeoutMs = __ov;
                } catch (e) { }
                await new Promise(function (resolve) {
                    let done = false;
                    const finish = function () { if (!done) { done = true; resolve(); } };
                    try { __readyPromise.then(finish, finish); } catch (e) { finish(); }
                    try { setTimeout(finish, __timeoutMs); } catch (e) { finish(); }
                });
            }
            if (!((typeof window !== 'undefined') && window.__prescriptionsReady === true)) {
                try { window.__lastSerialGateStat = { ok: false, reason: 'not-ready' }; } catch (e) { }
                try { alert('处方数据仍在加载，请稍候再开方（避免生成重复处方号）。若长时间如此，请重新登录或刷新页
面。'); } catch (e) { }
                return null;
            }
            const today = new Date().toLocaleDateString('sv-SE');
            const todayPrefix = today.replace(/-/g, '').slice(2); // YYMMDD
            let stored = null;
            try { stored = localStorage.getItem('global_max_serial'); } catch (e) { }
// 如果 localStorage 为空，尝试从 IndexedDB 读取
            if (!stored) {
                try {
                    const dbData = await getFromIndexedDB('settings', 'global_max_serial');
                    if (dbData && dbData.value) {
                        stored = typeof dbData.value === 'string' ? dbData.value : JSON.stringify(dbData.value);
                        localStorage.setItem('global_max_serial', stored);
                    }
                } catch (e) {
                    console.error('从 IndexedDB 读取序号失败:', e);
                }
            }
// 如果 localStorage 为空，尝试从文件系统读取
            if (!stored && window.electronAPI && window.electronAPI.isElectron) {
                try {
                    const result = await window.electronAPI.getUserData('global_max_serial');
                    if (result && result.success && result.data !== null) {
                        stored = typeof result.data === 'string' ? result.data : JSON.stringify(result.data);
                        localStorage.setItem('global_max_serial', stored);
```

```
                }
            } catch (e) {
                console.error('从 Electron 文件系统读取序号失败:', e);
            }
        }
        if (stored) {
            let data = null;
            try { data = JSON.parse(stored); } catch (e) { data = null; }
            if (data && data.date === today) {
                return data.maxSerial;
            }
        }
// 日期不匹配或无数据，扫描历史处方确定当日最大序号
// 权威集优先 —— localStorage 镜像的写盘已延后到空闲时
// 刚取的号可能还没落盘；此时若只读镜像会算小 → 重复号。
// 内存里的处方全量集是权威（loadData 已置就绪闸，故此处必然可信），镜像仅兜底。
        let maxSerial = 0;
        try {
            let allPrescriptions = [];
            try {
                const im = (typeof window !== 'undefined') ? window.__prescriptionListForSerial : null;
                if (Array.isArray(im)) allPrescriptions = im;
            } catch (e) { }
            if (allPrescriptions.length === 0 &&
                typeof prescriptionHistory !== 'undefined' && Array.isArray(prescriptionHistory) &&
                prescriptionHistory.length > 0) {
                allPrescriptions = prescriptionHistory;
            }
            if (allPrescriptions.length === 0) {
                let savedPrescriptions = null;
                try { savedPrescriptions = localStorage.getItem('all_prescription_list'); } catch (e) { }
                if (savedPrescriptions) {
                    allPrescriptions = JSON.parse(savedPrescriptions);
                } else {
                    const dbData = await getFromIndexedDB('settings', 'all_prescription_list');
                    if (dbData && dbData.value) {
                        const data = typeof dbData.value === 'string' ? JSON.parse(dbData.value) : dbData.value;
                        if (Array.isArray(data)) {
                            allPrescriptions = data;
                        }
                    }
                }
            }
            if (!Array.isArray(allPrescriptions)) allPrescriptions = [];
// 找出当日已有的最大编号
            allPrescriptions.forEach(p => {
                if (p.prescriptionNo && p.prescriptionNo.startsWith(todayPrefix)) {
                    const seqStr = p.prescriptionNo.slice(6);
                    const seq = parseInt(seqStr);
                    if (!isNaN(seq) && seq > maxSerial) {
                        maxSerial = seq;
                    }
```

```
                    }
                });
            } catch (e) {
                console.error('扫描历史处方确定序号失败:', e);
            }
 // 落盘仍走 setGlobalMaxSerial（本页权威实现），
 // 测试沙箱里经 window.setGlobalMaxSerial 注入同一实现（收口点可观测）。
            if (typeof setGlobalMaxSerial === 'function') setGlobalMaxSerial(maxSerial);
            else if (typeof window !== 'undefined' && typeof window.setGlobalMaxSerial === 'function')
window.setGlobalMaxSerial(maxSerial);
            return maxSerial;
        }
 // 设置当日最大序号
        function setGlobalMaxSerial(maxSerial) {
            const today = new Date().toLocaleDateString('sv-SE');
            const data = { date: today, maxSerial: maxSerial };
            localStorage.setItem('global_max_serial', JSON.stringify(data));
 // 同时保存到 IndexedDB（所有环境都支持）
            saveToIndexedDB('settings', { key: 'global_max_serial', value: data }).catch(e => {
                console.error('保存序号到 IndexedDB 失败:', e);
            });
 // 同时保存到 Electron 文件系统
            if (IS_ELECTRON) {
                window.electronAPI.saveUserData('global_max_serial', data).catch(e => {
                    console.error('保存序号到文件系统失败:', e);
                });
            }
        }
        async function renumberHistoricalPrescriptions() {
 // 从 prescriptions store 读取全量处方
            let allPrescriptions = await getAllPrescriptionsFromDB();
            if (allPrescriptions.length === 0) {
                return;
            }
            let hasChanges = false;
            const dateSerialMap = {};
            allPrescriptions.forEach(p => {
                if (!p.prescriptionNo || p.prescriptionNo.length !== 8 || !/^\d{8}$/.test(p.prescriptionNo)) {
                    const dateStr = p.date || new Date(p.createdAt || Date.now()).toLocaleDateString('sv-SE');
                    const datePrefix = dateStr.replace(/-/g, '').slice(2);
                    if (!dateSerialMap[datePrefix]) {
                        dateSerialMap[datePrefix] = 0;
                    }
                    allPrescriptions.forEach(p2 => {
                        if (p2.prescriptionNo && p2.prescriptionNo.startsWith(datePrefix)) {
                            const seq = parseInt(p2.prescriptionNo.slice(6));
                            if (!isNaN(seq) && seq > dateSerialMap[datePrefix]) {
                                dateSerialMap[datePrefix] = seq;
                            }
                        }
                    });
                    hasChanges = true;
```

```
                }
            });
            if (!hasChanges) {
                return;
            }
            const sortedPrescriptions = [...allPrescriptions].sort((a, b) => {
                return (a.createdAt || 0) - (b.createdAt || 0);
            });
            sortedPrescriptions.forEach(p => {
                if (!p.prescriptionNo || p.prescriptionNo.length !== 8 || !/^\d{8}$/.test(p.prescriptionNo)) {
                    const dateStr = p.date || new Date(p.createdAt || Date.now()).toLocaleDateString('sv-SE');
                    const datePrefix = dateStr.replace(/-/g, '').slice(2);
                    dateSerialMap[datePrefix]++;
                    p.prescriptionNo = datePrefix + String(dateSerialMap[datePrefix]).padStart(2, '0');
                    const originalIndex = allPrescriptions.findIndex(p2 => p2.id === p.id);
                    if (originalIndex !== -1) {
                        allPrescriptions[originalIndex].prescriptionNo = p.prescriptionNo;
                    }
                }
            });
 // 批量更新编号到 prescriptions store
            await saveAllPrescriptionsToDB(allPrescriptions).catch(e => {
                console.error('批量更新编号到 prescriptions store 失败:', e);
            });
        }
        async function generatePrescriptionNo() {
            const now = new Date();
            const year = String(now.getFullYear()).slice(-2);
            const month = String(now.getMonth() + 1).padStart(2, '0');
            const day = String(now.getDate()).padStart(2, '0');
            const currentMax = await getGlobalMaxSerial();
            const nextSeq = currentMax + 1;
            const count = String(nextSeq).padStart(2, '0');
            return year + month + day + count;
        }
        async function generatePrescriptionNoForSave() {
            const currentMax = await getGlobalMaxSerial();
            const newSeq = currentMax + 1;
            setGlobalMaxSerial(newSeq);
            const now = new Date();
            const year = String(now.getFullYear()).slice(-2);
            const month = String(now.getMonth() + 1).padStart(2, '0');
            const day = String(now.getDate()).padStart(2, '0');
            return year + month + day + String(newSeq).padStart(2, '0');
        }
/* ========== 源文件：index.html（选药候选四级排序 + 金额联动计算） ========== */
        function selectSearchResult() {
            const dropdown = DOM_CACHE.medicineSearchDropdown;
            const selected = dropdown.querySelector('.search-item.selected');
            if (selected) selected.click();
        }
        function showSearchDropdown(element, query, column) {
```

```
// 立即设置位置（与云端 APP 一致，避免 debounce 延迟导致键盘弹出后位置变化）
            const dropdown = DOM_CACHE.medicineSearchDropdown;
            if (hideSearchTimer) { clearTimeout(hideSearchTimer); hideSearchTimer = null; }
            const rect = element.getBoundingClientRect();
            dropdown.style.left = rect.left + 'px';
            dropdown.style.top = (rect.bottom + 2) + 'px';
            dropdown.style.width = (rect.width) + 'px';
            if (searchDebounceTimer) clearTimeout(searchDebounceTimer);
            searchDebounceTimer = setTimeout(() => {
                if (!query.trim()) {
                    dropdown.style.display = 'none';
                    return;
                }
                let filtered = [];
                const sanitizedQuery = query.replace(/[<>\"\'&]/g, '').toLowerCase();
                const maxResults = 50;
                if (column === 'code') {
                    if (medicineCodeIndex.size > 0) {
                        for (const [code, meds] of medicineCodeIndex) {
                            if (code.startsWith(sanitizedQuery)) {
                                filtered.push(...meds);
                                if (filtered.length >= maxResults) break;
                            }
                        }
                    } else {
                        for (const m of medicines) {
                            if (m.code && m.code.toLowerCase().startsWith(sanitizedQuery)) {
                                filtered.push(m);
                                if (filtered.length >= maxResults) break;
                            }
                        }
                    }
                } else {
                    for (const m of medicines) {
                        if (m.name && m.name.includes(sanitizedQuery)) {
                            filtered.push(m);
                            if (filtered.length >= maxResults) break;
                        }
                    }
                }
                const existingNames = new Set(prescriptionItems.map(item => item.name));
                filtered = filtered.filter(m => !existingNames.has(m.name));
                filtered.sort((a, b) => {
// 优先使用药品置顶（与离线 APP 统一）
                    const aPriority = a.priority_use ? 1 : 0;
                    const bPriority = b.priority_use ? 1 : 0;
                    if (aPriority !== bPriority) return bPriority - aPriority;
                    const aFreq = medicineFrequency[a.name] || 0;
                    const bFreq = medicineFrequency[b.name] || 0;
                    if (aFreq !== bFreq) return bFreq - aFreq;
                    const aNameLen = (a.name || '').length;
                    const bNameLen = (b.name || '').length;
```

```
                    if (aNameLen !== bNameLen) return aNameLen - bNameLen;
                    return (a.code || '').length - (b.code || '').length;
                });
                filtered = filtered.slice(0, 20);
                if (filtered.length >0) {
                    selectedSearchIndex = 0;
                    dropdown.innerHTML = filtered.map((m,i) =>{
                        const safeName = (m.name || '').replace(/[<>\"\'&]/g, '');
                        const safeCode = (m.code || '').replace(/[<>\"\'&]/g, '');
                        const safePrice = parseFloat(m.price) || 0;
                        const showDosage = m.dosage !== undefined && m.dosage !== null && m.dosage !== '' ?
parseInt(m.dosage) : 'undefined';
                        const priorityMark = m.priority_use ? '<span style="color:#e65100;font-weight:bold;"
title="优先使用">★</span> ' : '';
                        return `<div class="search-item ${i === selectedSearchIndex ? 'selected' : ''}"
${m.priority_use ? 'style="background:#fff8e1;"' : ''}
                                onclick="selectMedicine(${currentEditIndex},'${safeName}','${safeCode}',${safePrice}
)">${priorityMark}${safeName} [${safeCode}] - ${showDosage === 'undefined' ? '?' : showDosage + (m.unit ||
  'g')}${window.StockCore ? StockCore.dropBadge(m) : ''}</div>`;
                    }).join('');
                    dropdown.style.display = 'block';
 // 自适应位置：下方空间不足时自动翻到输入框上方显示（修复遮挡）
                    try {
                        const ddRect = dropdown.getBoundingClientRect();
                        const inputRect = element.getBoundingClientRect();
                        if (ddRect.bottom > window.innerHeight && inputRect.top > ddRect.height + 4) {
                            dropdown.style.top = Math.max(2, inputRect.top - ddRect.height - 2) + 'px';
                        }
                    } catch(e) {}
                } else {
                    dropdown.style.display = 'none';
                }
            }, 150);
        }
        function tryHideSearch() {
            if (hideSearchTimer) clearTimeout(hideSearchTimer);
            hideSearchTimer = setTimeout(() =>{DOM_CACHE.medicineSearchDropdown.style.display = 'none';}, 200);
        }
        function selectMedicine(index, name, code, price) {
            const exists = prescriptionItems.some((item, i) =>i !== index && item.name === name);
            if (exists) {
                alert('该药物已在处方中，不能重复添加！');
                DOM_CACHE.medicineSearchDropdown.style.display = 'none';
                return;
            }
            const medicine = getMedicineByName(name);
            const actualDosage = medicine && medicine.dosage !== undefined && medicine.dosage !== null &&
medicine.dosage !== '' ? medicine.dosage : 10;
            prescriptionItems[index].name = name;
            prescriptionItems[index].code = code;
            const __medSafePrice = isNaN(parseFloat(price)) ? 0 : parseFloat(price);
            prescriptionItems[index].price = __medSafePrice.toFixed(2);
```

```
            prescriptionItems[index].dosage = actualDosage;
            prescriptionItems[index].jianfa = getEffectiveJianfa(medicine, name);
            prescriptionItems[index].total = (__medSafePrice * actualDosage).toFixed(2);
            if (index >= prescriptionItems.length - 3) addEmptyRow();
            DOM_CACHE.init();
            renderPrescriptionTable();
            calculateAmount();
            updatePrescriptionPaper();
            DOM_CACHE.medicineSearchDropdown.style.display = 'none';
 // 选中即更新使用频率，让简码搜索按使用次数优先排序
            if (name) {
                medicineFrequency[name] = (medicineFrequency[name] || 0) + 1;
                saveMedicineFrequency();
            }
            const nextInput = document.getElementById(`code-input-${index + 1}`);
            if (nextInput) nextInput.focus();
            showToast(name + ' 已添加');
        }
        function showToast(message) {
            const toast = document.createElement('div');
            toast.style.cssText = 'position:fixed;top:50%;left:50%;transform:translate(-50%,-50%);background:#008000
;color:white;padding:10px 20px;border-radius:5px;font-size:12px;z-index:9999;opacity:0;transition:opacity 0.3s;';
            toast.textContent = message;
            document.body.appendChild(toast);
            setTimeout(() =>toast.style.opacity = '1', 10);
            setTimeout(() =>{
                toast.style.opacity = '0';
                setTimeout(() =>toast.remove(), 300);
            }, 3000);
        }
        function updateDosage(index, val) {
            const __medDose = parseFloat(val) || 10;
            prescriptionItems[index].dosage = __medDose;
            const __medUpPrice = parseFloat(prescriptionItems[index].price);
            const __medUpSafePrice = isNaN(__medUpPrice) ? 0 : __medUpPrice;
            prescriptionItems[index].total = (__medUpSafePrice * __medDose).toFixed(2);
 // 更新合计列显示
            const row = document.querySelector(`#prescriptionBody tr:nth-child(${index + 1})`);
            if (row) {
                const totalCell = row.querySelector('td:nth-child(7) input');
                if (totalCell) totalCell.value = prescriptionItems[index].total;
            }
            calculateAmount();
            updatePrescriptionPaper();
        }
        function updateUnit(index, val) {
            prescriptionItems[index].unit = val;
            updatePrescriptionPaper();
        }
        function deleteRow(index) {
            prescriptionItems.splice(index, 1);
            if (prescriptionItems.length< 15) addEmptyRow();
```

```
            DOM_CACHE.init();
            renderPrescriptionTable();
            calculateAmount();
            updatePrescriptionPaper();
        }
// 一键清空全部药物：点击表头 🗑 清空按钮触发，二次确认防误触
        function deleteAllDrugs() {
            const hasRealDrugs = prescriptionItems.some(item => item && item.name && item.name.trim());
            if (!hasRealDrugs) { alert('药物栏已经是空的'); return; }
            if (!confirm('确定要清空药物栏全部药物吗？此操作不可撤销。')) return;
            prescriptionItems = [];
            for (let i = 0; i < 15; i++) {
                prescriptionItems.push({ code: '', name: '', dosage: '', unit: 'g', price: '', total: '' });
            }
            DOM_CACHE.init();
            renderPrescriptionTable();
            calculateAmount();
            updatePrescriptionPaper();
        }
        function syncDoseInputs(sourceId) {
// 修复剂数无法修改：原版 parseInt('')||7 在用户删除数字时立即把默认值写回输入框，
// 数字永远删不掉改不了。空值中间态不回写输入框，仅显示/计算用兜底值。
            const raw = DOM_CACHE.doseCountInput3 ? String(DOM_CACHE.doseCountInput3.value) : '';
            const value = parseInt(raw) || 7;
            const display = document.getElementById('paperDoseCount');
            if (display) display.textContent = value;
            var _ud = document.getElementById('paperUsageDose'); if (_ud) _ud.textContent = value;
            calculateAmount();
        }
        function syncRegistrationFee(sourceId) {
// 修复诊疗费无法修改：原版 parseFloat('')||0 在用户删除数字时立即回写 0，
// 数字无法修改；现空值中间态不回写输入框（calculateAmount 内部有 ||'0' 兜底）。
            calculateAmount();
        }
// 剂数/诊疗费失焦兜底：用户清空后未输入新值直接离开时恢复默认值（7/0）
// 并刷新显示，避免输入框长期留空（事件委托一次绑定，不改 DOM 不改 CSS）
        document.addEventListener('focusout', function(e) {
            const t = e.target;
            if (!t || !t.id) return;
            if (t.id === 'doseCountInput3' && String(t.value).trim() === '') {
                t.value = 7;
                syncDoseInputs(t.id);
            } else if (t.id === 'registrationFee2' && String(t.value).trim() === '') {
                t.value = 0;
                syncRegistrationFee('registrationFee2');
            }
        });
        function calculateAmount() {
            const medicineTotal = prescriptionItems.reduce((sum, item) =>sum + (parseFloat(item.total) || 0), 0);
            const doseCount = parseInt(DOM_CACHE.doseCountInput3?.value || '7') || 7;
            const registrationFee = parseFloat(DOM_CACHE.registrationFee2?.value || '0') || 0;
            const total = medicineTotal * doseCount + registrationFee;
```

```
            const perDoseText = medicineTotal.toFixed(2);
            const totalText = (medicineTotal * doseCount).toFixed(2);
            const grandText = total.toFixed(2);
            const tempAmountEl = document.getElementById('tempAmount');
            if (tempAmountEl) tempAmountEl.value = grandText;
            if (DOM_CACHE.perDosePrice2) DOM_CACHE.perDosePrice2.textContent = perDoseText;
            if (DOM_CACHE.totalPrice2) DOM_CACHE.totalPrice2.textContent = totalText;
            if (DOM_CACHE.grandTotal2) DOM_CACHE.grandTotal2.textContent = grandText;
        }
/* ========== 源文件：symptom-dict.js（中医症状体征词典 / 跨类拼接 / 光标缓存 / Alt+S） ========== */
// ================================================================================
// symptom-dict.js — 中医症状体征词典 + 舌脉快捷录入面板（权威源）
// 设计说明：症状快捷录入模块（v1.0 定稿）
// 分发：随业务模块清单一并发布
// 缓存：静态资源缓存一天；词典修订需提升版本号以刷新缓存
// 特性：
// 1) 词条按《中医诊断学》标准内置（80~120 条），预置拼音简码
// 2) 快捷面板：分类点选 + 简码搜索（startsWith）/中文搜索（includes）
// 3) 拼接规则（§3.4）：同分类用顿号「、」，跨分类用逗号「，」，末尾无标点
// 4) 光标缓存插入（§4.2）：面板打开时缓存 selectionStart/End，插入后焦点回归
// 5) 频次记忆（§4.6）：localStorage[SYMPTOM_FREQ_KEY]，高频词前置，可重置
// 6) Alt+S 快捷键（§4.7）；「舌脉」按钮运行时注入 .history-tabs（HTML 零改动）
// ================================================================================
(function (global) {
    'use strict';
 // 防重复注入（script 标签 + 热更新场景双保险）
    if (global.__symptomDictLoaded) return;
    global.__symptomDictLoaded = true;
 // ============================================================================
 // 1. 词典数据（纯数据，无 DOM 依赖）
 // ============================================================================
    var DICT = {
        version: 2,
        categories: [
 // ---- 组合模板（order 0：快捷整串，追加不替换，§4.5）----
            { id: 'zh', name: '组合模板', order: 0, terms: [
                { text: '舌淡红，苔薄白，脉弦', code: '' },
                { text: '舌淡红，苔薄白，脉缓', code: '' },
                { text: '舌淡红，苔薄白，脉平', code: '' },
                { text: '舌红，苔黄腻，脉滑数', code: '' },
                { text: '舌红，苔薄黄，脉浮数', code: '' },
                { text: '舌淡，苔白腻，脉濡缓', code: '' },
                { text: '舌淡，苔薄白，脉沉细', code: '' },
                { text: '舌淡胖有齿痕，苔白滑，脉沉迟', code: '' },
                { text: '舌紫暗有瘀斑，苔薄白，脉涩', code: '' },
                { text: '舌红少苔，脉细数', code: '' },
                { text: '舌红绛，苔黄燥，脉洪数', code: '' },
                { text: '舌淡，苔白滑，脉紧', code: '' }
            ]},
 // ---- 舌质（order 1）----
            { id: 'tz', name: '舌质', order: 1, terms: [
                { text: '舌淡', code: 'sd' },
```

```
                { text: '舌淡红', code: 'sdh' },
                { text: '舌红', code: 'sh' },
                { text: '舌绛', code: 'sj' },
                { text: '舌红绛', code: 'shj' },
                { text: '舌紫暗', code: 'sza' },
                { text: '舌淡胖', code: 'sdp' },
                { text: '舌胖大', code: 'spd' },
                { text: '边有齿痕', code: 'bych' },
                { text: '舌有瘀斑', code: 'syyb' },
                { text: '舌有瘀点', code: 'syyd' },
                { text: '舌有裂纹', code: 'sylw' },
                { text: '舌瘦薄', code: 'ssb' },
                { text: '舌淡白', code: 'sdb' },
                { text: '舌歪斜', code: 'swx' }
            ]},
// ---- 舌苔（order 2）----
            { id: 'tai', name: '舌苔', order: 2, terms: [
                { text: '苔薄白', code: 'tbb' },
                { text: '苔薄黄', code: 'tbh' },
                { text: '苔白腻', code: 'tbn' },
                { text: '苔白厚', code: 'tbho' },
                { text: '苔白滑', code: 'tbai' },
                { text: '苔白干', code: 'tbg' },
                { text: '苔白润', code: 'tbr' },
                { text: '苔黄腻', code: 'thn' },
                { text: '苔黄厚', code: 'thho' },
                { text: '苔黄燥', code: 'thz' },
                { text: '苔灰黑', code: 'thh' },
                { text: '苔少', code: 'ts' },
                { text: '无苔', code: 'wt' },
                { text: '苔剥落', code: 'tbl' },
                { text: '花剥苔', code: 'hbt' }
            ]},
// ---- 脉象（order 3）----
            { id: 'mai', name: '脉象', order: 3, terms: [
                { text: '脉浮', code: 'mf' },
                { text: '脉沉', code: 'mc' },
                { text: '脉迟', code: 'mch' },
                { text: '脉数', code: 'msh' },
                { text: '脉滑', code: 'mh' },
                { text: '脉涩', code: 'mse' },
                { text: '脉弦', code: 'mx' },
                { text: '脉细', code: 'mxi' },
                { text: '脉弱', code: 'mr' },
                { text: '脉紧', code: 'mj' },
                { text: '脉洪', code: 'mhong' },
                { text: '脉濡', code: 'mru' },
                { text: '脉缓', code: 'mhuan' },
                { text: '脉促', code: 'mcu' },
                { text: '脉结', code: 'mjie' },
                { text: '脉代', code: 'md' }
            ]},
```

```
// ---- 望闻诊（order 4）----
          { id: 'wen', name: '望闻诊', order: 4, terms: [
              { text: '面色萎黄', code: 'mswh' },
              { text: '面色苍白', code: 'mscb' },
              { text: '面色潮红', code: 'msch' },
              { text: '面色青紫', code: 'msqz' },
              { text: '面色晦暗', code: 'msha' },
              { text: '神疲乏力', code: 'spfl' },
              { text: '形体消瘦', code: 'xtxs' },
              { text: '形体肥胖', code: 'xtfp' },
              { text: '语声低微', code: 'ysdw' },
              { text: '少气懒言', code: 'sqly' },
              { text: '气短', code: 'qd' },
              { text: '咳嗽', code: 'ks' },
              { text: '咳痰稀白', code: 'ktxb' },
              { text: '端坐呼吸', code: 'dzpx' },
              { text: '口唇紫暗', code: 'kcza' },
              { text: '口唇淡白', code: 'kcdb' },
              { text: '咽喉肿痛', code: 'yhzt' }
          ]},
// ---- 问诊（order 5，十问歌）----
          { id: 'wd', name: '问诊', order: 5, terms: [
              { text: '恶寒发热', code: 'ehfr' },
              { text: '恶寒', code: 'eh' },
              { text: '发热', code: 'fr' },
              { text: '寒热往来', code: 'hrwl' },
              { text: '午后潮热', code: 'whch' },
              { text: '自汗', code: 'zh' },
              { text: '盗汗', code: 'dh' },
              { text: '无汗', code: 'wh' },
              { text: '头痛', code: 'tt' },
              { text: '头晕', code: 'ty' },
              { text: '身重', code: 'sz' },
              { text: '关节痛', code: 'gjt' },
              { text: '腰痛', code: 'yt' },
              { text: '腰膝酸软', code: 'yxsr' },
              { text: '肢体麻木', code: 'ztmm' },
              { text: '便秘', code: 'bm' },
              { text: '大便溏薄', code: 'dbtb' },
              { text: '泄泻', code: 'xx' },
              { text: '小便短赤', code: 'xbdc' },
              { text: '小便清长', code: 'xbqz' },
              { text: '小便频数', code: 'xbps' },
              { text: '纳呆', code: 'nd' },
              { text: '纳差', code: 'nc' },
              { text: '多食易饥', code: 'dsyj' },
              { text: '口干', code: 'kg' },
              { text: '口苦', code: 'kk' },
              { text: '口淡', code: 'kd' },
              { text: '渴喜冷饮', code: 'kxly' },
              { text: '失眠', code: 'sm' },
              { text: '多梦', code: 'dm' },
```

```
                { text: '入睡困难', code: 'rskn' },
                { text: '嗜睡', code: 'sshu' },
                { text: '胸闷', code: 'xm' },
                { text: '胸痛', code: 'xtong' },
                { text: '心悸', code: 'xj' },
                { text: '胁肋胀痛', code: 'xlzt' },
                { text: '脘腹胀满', code: 'wbzm' },
                { text: '腹痛', code: 'ft' },
                { text: '腹胀', code: 'fz' },
                { text: '嗳气', code: 'aq' },
                { text: '泛酸', code: 'fs' },
                { text: '恶心', code: 'ex' },
                { text: '呕吐', code: 'ot' },
                { text: '耳鸣', code: 'em' },
                { text: '鼻塞流涕', code: 'bslt' }
            ]}
        ]
    };
// ========================================================================
// 2. 数据访问 API（供测试/铁闸/未来扩展调用）
// ========================================================================
    var SYMPTOM_FREQ_KEY = 'symptom_freq_v1';
    var _catIndex = {};   // text -> catId
    var _orderIndex = {}; // catId -> order
    (function buildIndex() {
        for (var ci = 0; ci < DICT.categories.length; ci++) {
            var cat = DICT.categories[ci];
            _orderIndex[cat.id] = cat.order;
            for (var ti = 0; ti < cat.terms.length; ti++) {
                _catIndex[cat.terms[ti].text] = cat.id;
            }
        }
    })();
    function search(q) {
        q = String(q || '').trim().toLowerCase();
        if (!q) return [];
        var out = [];
        for (var ci = 0; ci < DICT.categories.length; ci++) {
            var terms = DICT.categories[ci].terms;
            for (var ti = 0; ti < terms.length; ti++) {
                var t = terms[ti];
// §3.3：中文 includes 包含匹配；简码 startsWith 前缀匹配（小写比较）
                if (t.text.indexOf(q) >= 0 ||
                    (t.code && t.code.toLowerCase().indexOf(q) === 0)) {
                    out.push({ text: t.text, code: t.code, cat: DICT.categories[ci].id });
                }
            }
        }
        return out;
    }
// ========================================================================
// 3. 面板组件（运行时注入，HTML 静态结构零改动）
```

```
// ========================================================================
    var _symCursor = null;          // 光标缓存（§4.2 R1）
    var _selected = [];             // 已选词条 [{text, cat}]
    var _activeCat = 'zh';          // 当前分类
    var _freq = {};                 // 频次 {text: count}
    function loadFreq() {
        try {
            var raw = global.localStorage ? global.localStorage.getItem(SYMPTOM_FREQ_KEY) : null;
            if (raw) {
                var obj = JSON.parse(raw);
                if (obj && typeof obj === 'object') _freq = obj;
            }
        } catch (e) { _freq = {}; }
    }
    function saveFreq() {
        try {
            if (global.localStorage) global.localStorage.setItem(SYMPTOM_FREQ_KEY, JSON.stringify(_freq));
        } catch (e) { /* 存储失败不影响主流程 */ }
    }
// §3.4 拼接：按分类 order 升序分组 → 组内「、」、组间「，」、末尾无标点
    function assembleText(selected) {
        var groups = {}, catIds = [];
        selected.slice().sort(function (a, b) {
// 注意：order=0 是合法值，不可用 || 兜底（falsy 陷阱）
            var oa = (a.cat in _orderIndex) ? _orderIndex[a.cat] : 99;
            var ob = (b.cat in _orderIndex) ? _orderIndex[b.cat] : 99;
            return oa - ob;
        }).forEach(function (t) {
            if (!groups[t.cat]) { groups[t.cat] = []; catIds.push(t.cat); }
            groups[t.cat].push(t.text);
        });
        return catIds.map(function (c) { return groups[c].join('、'); }).join('，');
    }
// §4.2 光标缓存插入
    function insertAtCursor(text) {
        var ta = document.getElementById('medicalHistory');
        if (!ta || !text) return;
        var pos = _symCursor || { start: ta.value.length, end: ta.value.length };
        var before = ta.value.slice(0, pos.start);
// 智能补逗号：插入点前已有内容且前一字符不是标点/换行时，前置「，」
        var needComma = before.length > 0 && !/[，、,;；。\n]$/.test(before);
        var insertText = (needComma ? '，' : '') + text;
        ta.value = before + insertText + ta.value.slice(pos.end);
        var newPos = pos.start + insertText.length;
        try { ta.setSelectionRange(newPos, newPos); } catch (e) { /* 兼容旧内核 */ }
        ta.focus();
        _symCursor = { start: newPos, end: newPos }; // 支持连续多次插入
// 同步处方笺预览（防御式跨 IIFE 调用，铁律）
        if (typeof global.updatePrescriptionPaper === 'function') global.updatePrescriptionPaper();
        else if (typeof updatePrescriptionPaper === 'function') updatePrescriptionPaper();
    }
    function openSymptomPanel() {
```

```
        var ta = document.getElementById('medicalHistory');
        if (!ta) { alert('症状输入框未找到'); return; }
// ① 第一时间缓存光标（弹窗抢焦点会导致 selection 重置，R1）
        _symCursor = { start: ta.selectionStart, end: ta.selectionEnd };
        ensurePanelDom();
        renderChips();
        renderTerms();
        renderPreview();
        if (typeof global.showModal === 'function') global.showModal('symptomModal');
        else if (typeof showModal === 'function') showModal('symptomModal');
        else {
            var m = document.getElementById('symptomModal');
            if (m) m.style.display = 'flex';
        }
    }
    function closeSymptomPanel() {
        if (typeof global.closeModal === 'function') global.closeModal('symptomModal');
        else if (typeof closeModal === 'function') closeModal('symptomModal');
        else {
            var m = document.getElementById('symptomModal');
            if (m) m.style.display = 'none';
        }
// §4.4/Y5：焦点回归症状输入框，医生可无缝继续手动输入
        var ta = document.getElementById('medicalHistory');
        if (ta && _symCursor) {
            try { ta.setSelectionRange(_symCursor.start, _symCursor.end); } catch (e) {}
            ta.focus();
        } else if (ta) ta.focus();
    }
    function confirmInsert() {
        if (!_selected.length) { showToastTip('请先选择词条'); return; }
        insertAtCursor(assembleText(_selected));
// 频次记忆（§4.6）
        _selected.forEach(function (t) { _freq[t.text] = (_freq[t.text] || 0) + 1; });
        saveFreq();
        _selected = [];
        closeSymptomPanel();
    }
// ---------- DOM 注入 ----------
    var _domReady = false;
    function injectStyle() {
        if (document.getElementById('symptomPanelStyle')) return;
        var st = document.createElement('style');
        st.id = 'symptomPanelStyle';
        st.textContent =
            '#symptomModal .modal-content{max-width:500px;width:92%;max-height:84vh;display:flex;flex-direction:colu
mn;}' +
            '#symptomSearch{width:100%;padding:6px 8px;border:1px solid
#808080;border-radius:3px;font-size:13px;box-sizing:border-box;}' +
            '.sym-chips{display:flex;flex-wrap:wrap;gap:4px;margin:8px 0;}' +
            '.sym-chip{padding:3px 10px;border:1px solid
#808080;border-radius:10px;background:#f0f0f0;cursor:pointer;font-size:11px;}' +
```

```
            '.sym-chip.on{background:#008000;color:#fff;border-color:#008000;}' +
            '.sym-terms{display:flex;flex-wrap:wrap;gap:5px;max-height:180px;overflow-y:auto;padding:2px;align-conte
nt:flex-start;-webkit-overflow-scrolling:touch;}' +
            '.sym-term{padding:3px 10px;min-height:28px;border:1px solid
#a0a0a0;border-radius:3px;background:#fff;cursor:pointer;font-size:12px;line-height:1.4;}' +
            '.sym-term:active{background:#e8e8e8;}' +
            '.sym-term.on{background:#e8f5e9;border-color:#008000;color:#008000;font-weight:bold;}' +
            '.sym-preview{margin-top:8px;padding:6px 8px;border:1px dashed
#808080;border-radius:3px;background:#fafafa;font-size:12px;line-height:1.5;min-height:30px;}' +
            '.sym-preview .sym-pv-label{color:#666;}' +
            '.sym-preview .sym-pv-text{color:#006000;font-weight:bold;word-break:break-all;}' +
            '.sym-tip{margin-top:6px;font-size:10px;color:#999;}' +
            '.sym-mini-btn{font-size:10px;padding:1px 8px;border:1px solid
#bbb;border-radius:8px;background:#fff;cursor:pointer;color:#666;}' +
            '@media screen and (max-width:768px){' +
            '  .sym-term{min-height:36px;font-size:13px;padding:5px 12px;}' + // Y3 触摸目标 ≥36px
            '  .sym-chip{min-height:32px;padding:5px 12px;font-size:12px;}' +
            '  .sym-terms{max-height:200px;}' +
            '}';
        document.head.appendChild(st);
    }
    function injectButton() {
        var tabs = document.querySelector('.history-tabs');
        if (!tabs || document.getElementById('symptomQuickBtn')) return false;
        var btn = document.createElement('div');
        btn.id = 'symptomQuickBtn';
        btn.className = 'history-tab';
        btn.style.background = '#e8f5e9';
        btn.style.color = '#008000';
        btn.style.marginLeft = 'auto';
        btn.style.fontWeight = 'bold';
        btn.textContent = '舌脉';
        btn.title = '舌脉体征快捷录入（Alt+S）';
        btn.addEventListener('click', function (e) {
            e.preventDefault();
            e.stopPropagation();
            openSymptomPanel();
        });
        tabs.appendChild(btn);
        return true;
    }
    function injectModal() {
        if (document.getElementById('symptomModal')) return;
        var m = document.createElement('div');
        m.id = 'symptomModal';
        m.className = 'modal';
        m.style.display = 'none';
        m.innerHTML =
            '<div class="modal-content">' +
            '  <div class="modal-header"><h3>舌脉体征快捷录入</h3><span class="close-btn"
id="symptomCloseBtn">&times;</span></div>' +
            '  <div class="modal-body">' +
```

```
            '    <input type="text" id="symptomSearch" placeholder="搜索名称或拼音简码：sd=舌淡，mx=脉弦"
autocomplete="off">' +
            '    <div class="sym-chips" id="symptomChips"></div>' +
            '    <div id="symptomTemplateTip" style="display:none;font-size:10px;color:#999;margin-bottom:4px;">模板
为整串插入，不清空已选</div>' +
            '    <div class="sym-terms" id="symptomTerms"></div>' +
            '    <div class="sym-preview" id="symptomPreview"></div>' +
            '    <div class="sym-tip">点选词条快速录入，支持拼音简码搜索，Alt+S 快捷唤起 <button type="button"
class="sym-mini-btn" id="symptomFreqReset">重置频次</button></div>' +
            '  </div>' +
            '  <div class="modal-footer">' +
            '    <button class="action-btn" id="symptomCancelBtn">取消</button>' +
            '    <button class="action-btn primary" id="symptomOkBtn">确认插入</button>' +
            '  </div>' +
            '</div>';
        document.body.appendChild(m);
        document.getElementById('symptomCloseBtn').addEventListener('click', closeSymptomPanel);
        document.getElementById('symptomCancelBtn').addEventListener('click', closeSymptomPanel);
        document.getElementById('symptomOkBtn').addEventListener('click', confirmInsert);
        document.getElementById('symptomFreqReset').addEventListener('click', function () {
            if (confirm('确定重置词条使用频次吗？')) {
                _freq = {}; saveFreq(); renderTerms(); showToastTip('频次已重置');
            }
        });
        document.getElementById('symptomSearch').addEventListener('input', function () {
            renderTerms();
        });
 // 点击遮罩关闭（与现有 modal 行为一致由 closeModal 管理，此处兜底）
        m.addEventListener('click', function (e) { if (e.target === m) closeSymptomPanel(); });
    }
    function ensurePanelDom() {
        injectStyle();
        injectModal();
    }
 // ---------- 渲染 ----------
    function freqSorted(terms) {
 // §4.6：分类内按频次降序（stable，同频次保持词典原序）
        return terms.slice().sort(function (a, b) {
            return (_freq[b.text] || 0) - (_freq[a.text] || 0);
        });
    }
    function renderChips() {
        var box = document.getElementById('symptomChips');
        if (!box) return;
        var html = '';
        var cats = DICT.categories.slice().sort(function (a, b) { return a.order - b.order; });
        for (var i = 0; i < cats.length; i++) {
            html += '<div class="sym-chip' + (cats[i].id === _activeCat ? ' on' : '') + '" data-cat="' + cats[i].id
+ '">' + cats[i].name + '</div>';
        }
        box.innerHTML = html;
    }
```

```
    function renderTerms() {
        var box = document.getElementById('symptomTerms');
        if (!box) return;
        var searchEl = document.getElementById('symptomSearch');
        var q = searchEl ? searchEl.value.trim() : '';
        var list = [];
        if (q) {
            list = search(q); // 跨分类搜索结果
        } else {
            var cat = null;
            for (var i = 0; i < DICT.categories.length; i++) {
                if (DICT.categories[i].id === _activeCat) { cat = DICT.categories[i]; break; }
            }
            if (cat) list = freqSorted(cat.terms).map(function (t) { return { text: t.text, code: t.code, cat:
cat.id }; });
        }
        var tip = document.getElementById('symptomTemplateTip');
        if (tip) tip.style.display = (!q && _activeCat === 'zh') ? 'block' : 'none';
        if (!list.length) {
            box.innerHTML = '<span style="font-size:11px;color:#999;padding:4px;">无匹配词条</span>';
            return;
        }
        var html = '';
        for (var j = 0; j < list.length; j++) {
            var sel = _selected.some(function (s) { return s.text === list[j].text; });
            var codeTip = list[j].code ? ' title="简码:' + list[j].code + '"' : '';
            html += '<div class="sym-term' + (sel ? ' on' : '') + '" data-text="' + list[j].text + '"' + codeTip +
'>' + list[j].text + '</div>';
        }
        box.innerHTML = html;
    }
    function renderPreview() {
        var pv = document.getElementById('symptomPreview');
        if (!pv) return;
        if (!_selected.length) {
            pv.innerHTML = '<span class="sym-pv-label">已选：（点击词条试试，可连续多选、再点取消）</span>';
            return;
        }
        pv.innerHTML = '<span class="sym-pv-label">将插入：</span><span class="sym-pv-text">' +
assembleText(_selected) + '</span>';
    }
    function showToastTip(msg) {
        if (typeof global.showToast === 'function') global.showToast(msg);
        else if (typeof showToast === 'function') showToast(msg);
        else alert(msg);
    }
 // ---------- 事件委托 ----------
    function bindPanelEvents() {
        var terms = document.getElementById('symptomTerms');
        var chips = document.getElementById('symptomChips');
        if (terms && !terms.__symBound) {
            terms.__symBound = true;
```

```
            terms.addEventListener('click', function (e) {
                var el = e.target.closest ? e.target.closest('.sym-term') : null;
                if (!el) return;
                var text = el.getAttribute('data-text');
                var catId = _catIndex[text] || 'zh';
                var idx = -1;
                for (var i = 0; i < _selected.length; i++) { if (_selected[i].text === text) { idx = i; break; } }
                if (idx >= 0) _selected.splice(idx, 1); // 反选
                else _selected.push({ text: text, cat: catId }); // 追加（Y6：模板追加不替换）
                renderTerms(); renderPreview();
            });
        }
        if (chips && !chips.__symBound) {
            chips.__symBound = true;
            chips.addEventListener('click', function (e) {
                var el = e.target.closest ? e.target.closest('.sym-chip') : null;
                if (!el) return;
                _activeCat = el.getAttribute('data-cat');
                var searchEl = document.getElementById('symptomSearch');
                if (searchEl) searchEl.value = ''; // 切分类清空搜索
                renderChips(); renderTerms();
            });
        }
    }
// ---------- 自举 ----------
    function tryInit(retry) {
        retry = retry || 0;
        if (_domReady) return;
        var ok = injectButton();
        if (ok) {
            _domReady = true;
            injectStyle();
            injectModal();
            bindPanelEvents();
            loadFreq();
            return;
        }
        if (retry < 40) { // 40 x 250ms = 10s 兜底（等 SPA 场景 DOM 就绪）
            setTimeout(function () { tryInit(retry + 1); }, 250);
        }
    }
    if (document.readyState === 'loading') {
        document.addEventListener('DOMContentLoaded', function () { tryInit(0); });
    } else {
        tryInit(0);
    }
// Alt+S 快捷键（§4.7：避开 Alt+1 辨证选方、F1~F9）
    document.addEventListener('keydown', function (e) {
        if (e.altKey && (e.key === 's' || e.key === 'S')) {
            var m = document.getElementById('symptomModal');
            if (m && m.style.display !== 'none' && m.style.display !== '') {
                closeSymptomPanel();
```

```
            } else {
                e.preventDefault();
                openSymptomPanel();
            }
        }
    });
 // ============================================================================
 // 4. 导出（供测试与 P3 自定义词条扩展）
 // ============================================================================
    global.SYMPTOM_DICT = DICT;
    global.SymptomDict = {
        version: DICT.version,
        search: search,
        assembleText: assembleText,
        openPanel: openSymptomPanel,
        closePanel: closeSymptomPanel,
        getFreq: function () { return JSON.parse(JSON.stringify(_freq)); }
    };
})(typeof window !== 'undefined' ? window : this);
// ============================================================
// 4. 诊断快速输入模块（ 新增，权威源）
// 依据：《中医诊断学》教材 + GB/T 15657《中医病证分类与代码》
// ============================================================
(function (global) {
    'use strict';
    if (global.__diagQuickLoaded) return;
    global.__diagQuickLoaded = true;
    var DISEASE = [
        { t: '感冒', c: 'gm' }, { t: '时行感冒', c: 'sxgm' }, { t: '咳嗽', c: 'ks' },
        { t: '哮病', c: 'xbing' }, { t: '喘证', c: 'chuanzheng' }, { t: '肺胀', c: 'feizhang' },
        { t: '肺痈', c: 'feiyong' }, { t: '肺痨', c: 'feilao' }, { t: '肺痿', c: 'feiwei' },
        { t: '鼻渊', c: 'byuan' }, { t: '鼻窒', c: 'bzhi' }, { t: '喉痹', c: 'houbi' },
        { t: '乳蛾', c: 'rue' }, { t: '失音', c: 'shiyin' },
        { t: '心悸', c: 'xj' }, { t: '怔忡', c: 'zhengchong' }, { t: '胸痹', c: 'xiongb' },
        { t: '真心痛', c: 'zhenxintong' }, { t: '心衰', c: 'xinshuai' }, { t: '不寐', c: 'bumei' },
        { t: '多梦', c: 'duomeng' }, { t: '健忘', c: 'jianwang' }, { t: '狂病', c: 'kuangbing' },
        { t: '癫病', c: 'dianbing' }, { t: '痫病', c: 'xianbing' }, { t: '痴呆', c: 'chidai' },
        { t: '胃痛', c: 'weitong' }, { t: '痞满', c: 'piman' }, { t: '胃痞', c: 'weipi' },
        { t: '呕吐', c: 'outu' }, { t: '吐酸', c: 'tusuan' }, { t: '嘈杂', c: 'caoz' },
        { t: '噎膈', c: 'yege' }, { t: '呃逆', c: 'eni' }, { t: '腹痛', c: 'futong' },
        { t: '泄泻', c: 'xiexie' }, { t: '痢疾', c: 'liji' }, { t: '便秘', c: 'bianmi' },
        { t: '霍乱', c: 'huoluan' }, { t: '虫证', c: 'chongzheng' },
        { t: '胁痛', c: 'xietong' }, { t: '黄疸', c: 'huangdan' }, { t: '鼓胀', c: 'guzhang' },
        { t: '积聚', c: 'jiju' }, { t: '瘿病', c: 'yingbing' }, { t: '瘰疬', c: 'luoli' },
        { t: '疟疾', c: 'nueji' }, { t: '眩晕', c: 'xuanyun' }, { t: '头痛', c: 'toutong' },
        { t: '头风', c: 'toufeng' }, { t: '中风', c: 'zhongfeng' }, { t: '中风后遗症', c: 'zfyhz' },
        { t: '颤证', c: 'chanzheng' }, { t: '痉证', c: 'jingzheng' }, { t: '郁证', c: 'yuzheng' },
        { t: '水肿', c: 'shuizhong' }, { t: '淋证', c: 'linzheng' }, { t: '癃闭', c: 'longbi' },
        { t: '关格', c: 'guange' }, { t: '遗精', c: 'yijing' }, { t: '早泄', c: 'zaoxie' },
        { t: '阳痿', c: 'yangwei' }, { t: '不育', c: 'buyu' }, { t: '腰痛', c: 'yaotong' },
        { t: '遗尿', c: 'yiniao' }, { t: '尿浊', c: 'niaozhuo' },
```

```
    { t: '痰饮', c: 'tanyin' }, { t: '消渴', c: 'xiaoke' }, { t: '自汗', c: 'zihan' },
    { t: '盗汗', c: 'daohan' }, { t: '内伤发热', c: 'nsfr' }, { t: '虚劳', c: 'xulao' },
    { t: '血证', c: 'xuezheng' }, { t: '鼻衄', c: 'binv' }, { t: '咳血', c: 'kexue' },
    { t: '吐血', c: 'tuxue' }, { t: '便血', c: 'bianxue' }, { t: '尿血', c: 'niaoxue' },
    { t: '紫斑', c: 'ziban' }, { t: '肥胖', c: 'feipang' }, { t: '癌病', c: 'aibing' },
    { t: '厥证', c: 'juezheng' },
    { t: '痹证', c: 'bizheng' }, { t: '行痹', c: 'xingbi' }, { t: '痛痹', c: 'tongbi' },
    { t: '着痹', c: 'zhuobi' }, { t: '热痹', c: 'rebi' }, { t: '尪痹', c: 'wangbi' },
    { t: '痿证', c: 'weizheng' }, { t: '颈痹', c: 'jingbi' }, { t: '肩痹', c: 'jianbi' },
    { t: '跟痛症', c: 'gentongzheng' },
    { t: '疮疡', c: 'chuangyang' }, { t: '疖', c: 'jie' }, { t: '疔', c: 'ding' },
    { t: '痈', c: 'yong' }, { t: '疽', c: 'ju' }, { t: '发颐', c: 'fayi' },
    { t: '丹毒', c: 'dandu' }, { t: '瘿瘤', c: 'yingliu' }, { t: '乳癖', c: 'rupi' },
    { t: '乳痈', c: 'ruyong' }, { t: '乳岩', c: 'ruyan' }, { t: '湿疹', c: 'shizhen' },
    { t: '瘾疹', c: 'yinzhen' }, { t: '牛皮癣', c: 'niupixuan' }, { t: '白疕', c: 'baibi' },
    { t: '蛇串疮', c: 'shechuanchuang' }, { t: '油风', c: 'youfeng' }, { t: '粉刺', c: 'fenci' },
    { t: '黧黑斑', c: 'liheiban' }, { t: '冻疮', c: 'dongchuang' }, { t: '烧烫伤', c: 'shaotangshang' },
    { t: '破伤风', c: 'poshangfeng' }, { t: '精浊', c: 'jingzhuo' }, { t: '子痈', c: 'ziyong' },
    { t: '脱疽', c: 'tuoju' }, { t: '股肿', c: 'guzhong' },
    { t: '月经先期', c: 'yjxq' }, { t: '月经后期', c: 'yjhq' }, { t: '月经先后无定期', c: 'yjhxw' },
    { t: '月经过多', c: 'yjgd' }, { t: '月经过少', c: 'yjgs' }, { t: '经期延长', c: 'jqyc' },
    { t: '经间期出血', c: 'jjqc' }, { t: '崩漏', c: 'benglous' }, { t: '闭经', c: 'bijng' },
    { t: '痛经', c: 'tongjing' }, { t: '经行乳房胀痛', c: 'jxrfzt' }, { t: '经行头痛', c: 'jxtt' },
    { t: '经行发热', c: 'jxfr' }, { t: '经行泄泻', c: 'jxxx' }, { t: '绝经前后诸证', c: 'jjqhz' },
    { t: '带下过多', c: 'dxgd' }, { t: '带下过少', c: 'dxgs' }, { t: '妊娠恶阻', c: 'rsesz' },
    { t: '胎漏', c: 'tailou' }, { t: '胎动不安', c: 'tdba' }, { t: '滑胎', c: 'huatai' },
    { t: '子肿', c: 'zizhong' }, { t: '产后发热', c: 'chfr' }, { t: '产后腹痛', c: 'chft' },
    { t: '产后恶露不绝', c: 'clelbj' }, { t: '缺乳', c: 'queru' }, { t: '不孕症', c: 'buyunzheng' },
    { t: '癥瘕', c: 'zhengjia' }, { t: '阴痒', c: 'yinyang' }, { t: '阴挺', c: 'yinting' },
    { t: '肺炎喘嗽', c: 'fycsh' }, { t: '哮喘', c: 'xiaochuan' }, { t: '鹅口疮', c: 'ekouchuang' },
    { t: '口疮', c: 'kouchuang' }, { t: '厌食', c: 'yanshi' }, { t: '积滞', c: 'jizhi' },
    { t: '疳证', c: 'ganzheng' }, { t: '营养性缺铁性贫血', c: 'yytiepin' }, { t: '汗证', c: 'ethanzheng' },
    { t: '惊风', c: 'jingfeng' }, { t: '慢惊风', c: 'manjf' }, { t: '癫痫', c: 'dianxian' },
    { t: '急性肾小球肾炎', c: 'jxsxqy' }, { t: '肾病综合征', c: 'sbzhz' },
    { t: '五迟', c: 'wuci' }, { t: '五软', c: 'wuruan' }, { t: '麻疹', c: 'mazhen' },
    { t: '风痧', c: 'fengsha' }, { t: '丹痧', c: 'dansha' }, { t: '水痘', c: 'shuidou' },
    { t: '痄腮', c: 'zhasai' }, { t: '手足口病', c: 'szkb' }, { t: '顿咳', c: 'dunke' },
    { t: '小儿暑温', c: 'xeshuwen' }, { t: '疫毒痢', c: 'yiduli' },
    { t: '针眼', c: 'zhenyan' }, { t: '沙眼', c: 'shayan' }, { t: '目赤肿痛', c: 'muchztong' },
    { t: '圆翳内障', c: 'yyneizhang' }, { t: '脓耳', c: 'nonger' }, { t: '耳鸣', c: 'erming' },
    { t: '耳聋', c: 'erlong' }, { t: '鼻鼽', c: 'biqiu' }, { t: '牙痛', c: 'yatong' },
    { t: '牙宣', c: 'yaxuan' }
];
var SYNDROMES = [
    { t: '表证', c: 'bz' }, { t: '里证', c: 'lz' }, { t: '寒证', c: 'hzheng' },
    { t: '热证', c: 'rzheng' }, { t: '虚证', c: 'xuzheng' }, { t: '实证', c: 'shizheng' },
    { t: '阴证', c: 'yinzheng' }, { t: '阳证', c: 'yangzheng' },
    { t: '表寒', c: 'biaohan' }, { t: '表热', c: 'biaore' }, { t: '里寒', c: 'lihan' },
    { t: '里热', c: 'lire' }, { t: '虚寒', c: 'xuhan' }, { t: '虚热', c: 'xure' },
    { t: '实寒', c: 'shihan' }, { t: '实热', c: 'shire' },
    { t: '寒热错杂', c: 'hrcz' }, { t: '虚实夹杂', c: 'xszajia' }, { t: '表里同病', c: 'bltb' },
```

```
        { t: '亡阳', c: 'wangyang' }, { t: '亡阴', c: 'wangyin' },
        { t: '心气虚', c: 'xinqx' }, { t: '心血虚', c: 'xinxx' }, { t: '心阴虚', c: 'xinyinxu' },
        { t: '心阳虚', c: 'xinyangxu' }, { t: '心火亢盛', c: 'xhkangsheng' }, { t: '心血瘀阻', c: 'xxyuzu' },
        { t: '痰蒙心神', c: 'tanmengxinshen' }, { t: '痰火扰神', c: 'tanhuoraoshen' }, { t: '小肠实热', c:
'xiaochangshire' },
        { t: '肺气虚', c: 'feiqx' }, { t: '肺阴虚', c: 'feiyinxu' },
        { t: '风寒束肺', c: 'fhshufei' }, { t: '风热犯肺', c: 'frfanfei' }, { t: '燥邪犯肺', c: 'zaoxiefanfei' },
        { t: '肺热炽盛', c: 'frcsheng' }, { t: '痰湿阻肺', c: 'tanshizufei' },
        { t: '痰热壅肺', c: 'tanyongfei' }, { t: '寒饮停肺', c: 'hantingfei' },
        { t: '大肠湿热', c: 'dachangshire' }, { t: '肠燥津亏', c: 'changzaojinkui' },
        { t: '脾气虚', c: 'piqx' }, { t: '脾阳虚', c: 'piyangxu' }, { t: '脾虚气陷', c: 'pxqx' },
        { t: '脾不统血', c: 'pbutongxue' }, { t: '寒湿困脾', c: 'hanskpi' }, { t: '湿热蕴脾', c: 'shiryp' },
        { t: '胃气虚', c: 'weiqx' }, { t: '胃阴虚', c: 'weiyinxu' }, { t: '胃阳虚', c: 'weiyangxu' },
        { t: '胃火炽盛', c: 'weihuo' }, { t: '寒滞胃脘', c: 'hanzhiweiwan' }, { t: '食滞胃脘', c: 'shizhiww' },
        { t: '肝气郁结', c: 'gqyj' }, { t: '肝火炽盛', c: 'ghcsheng' }, { t: '肝阳上亢', c: 'gyshangkang' },
        { t: '肝风内动', c: 'gfnendong' }, { t: '肝阳化风', c: 'gyhuafeng' }, { t: '热极生风', c: 'rejisfeng' },
        { t: '阴虚动风', c: 'yinxfdongfeng' }, { t: '血虚生风', c: 'xuexsfeng' },
        { t: '肝血虚', c: 'ganxx' }, { t: '肝阴虚', c: 'ganyinxu' },
        { t: '肝胆湿热', c: 'gandanshire' }, { t: '寒凝肝脉', c: 'hnganmai' }, { t: '胆郁痰扰', c: 'danyutanrao' },
        { t: '肾阳虚', c: 'shenyangxu' }, { t: '肾阴虚', c: 'shenyinxu' }, { t: '肾精不足', c: 'shenjingbuzu' },
        { t: '肾气不固', c: 'shenqibugu' }, { t: '肾不纳气', c: 'shenbunaqi' }, { t: '肾虚水泛', c: 'shenxshuifan'
},
        { t: '膀胱湿热', c: 'pgshire' }, { t: '膀胱虚寒', c: 'pgxh' },
        { t: '太阳伤寒', c: 'tyshanghan' }, { t: '太阳中风', c: 'tyzhongfeng' },
        { t: '阳明经证', c: 'ymjingzheng' }, { t: '阳明腑证', c: 'ymfuzheng' },
        { t: '少阳证', c: 'shaoyangzheng' }, { t: '太阴证', c: 'taiyinzheng' },
        { t: '少阴寒化', c: 'shaoyinhanhua' }, { t: '少阴热化', c: 'shaoyinrehua' }, { t: '厥阴证', c:
'jueyinzheng' },
        { t: '卫分证', c: 'weifenzheng' }, { t: '气分证', c: 'qifenzheng' },
        { t: '营分证', c: 'yingfenzheng' }, { t: '血分证', c: 'xuefenzheng' },
        { t: '上焦病证', c: 'shangjiao' }, { t: '中焦病证', c: 'zhongjiao' }, { t: '下焦病证', c: 'xiajiao' },
        { t: '气虚', c: 'qx' }, { t: '气陷', c: 'qixian' }, { t: '气滞', c: 'qizhi' },
        { t: '气逆', c: 'qini' }, { t: '气闭', c: 'qibi' }, { t: '气脱', c: 'qituo' },
        { t: '血虚', c: 'xx' }, { t: '血瘀', c: 'xyu' }, { t: '血热', c: 'xre' }, { t: '血寒', c: 'xhan' },
        { t: '气血两虚', c: 'qxlx' }, { t: '气虚血瘀', c: 'qxxyu' }, { t: '气滞血瘀', c: 'qizhixyu' },
        { t: '气不摄血', c: 'qibushexue' }, { t: '气随血脱', c: 'qisuixuetuo' },
        { t: '津液亏虚', c: 'jykuixu' }, { t: '水湿停聚', c: 'sssjtiju' }, { t: '痰浊内阻', c: 'tanzuoneizu' },
        { t: '水饮内停', c: 'shuiyinting' },
        { t: '风淫', c: 'fengyin' }, { t: '寒淫', c: 'hanyin' }, { t: '暑淫', c: 'shuyin' },
        { t: '湿淫', c: 'shiyin' }, { t: '燥淫', c: 'zaoyin' }, { t: '火淫', c: 'huoyin' }
    ];
    var COMBOS = [
        { t: '感冒（风寒束表）', c: 'gmfh' }, { t: '感冒（风热犯表）', c: 'gmfr' },
        { t: '感冒（暑湿伤表）', c: 'gmss' },
        { t: '咳嗽（风寒袭肺）', c: 'ksfh' }, { t: '咳嗽（风热犯肺）', c: 'ksfr' },
        { t: '咳嗽（痰湿蕴肺）', c: 'ksts' }, { t: '咳嗽（痰热郁肺）', c: 'kstanyure' },
        { t: '咳嗽（肺阴亏虚）', c: 'ksfyxk' },
        { t: '哮病（冷哮）', c: 'xbl' }, { t: '哮病（热哮）', c: 'xbr' }, { t: '哮病（风痰哮）', c: 'xbft' }, { t:
'哮病（虚哮）', c: 'xbxx' },
        { t: '喘证（风寒壅肺）', c: 'czfh' }, { t: '喘证（表寒肺热）', c: 'czbfr' },
        { t: '喘证（痰热郁肺）', c: 'cztry' }, { t: '喘证（痰浊阻肺）', c: 'cztzz' },
```

```
        var arr = getMedArrayLive();
        if (arr) {
            for (var i = 0; i < arr.length; i++) {
                if (arr[i] && arr[i].name === name) {
                    arr[i].stock = round2((parseFloat(arr[i].stock) || 0) + delta);
                    persistList(arr);
                    return arr[i].stock;
                }
            }
            return null; // 活数组中无此药（已删/改名）：不再降级，防双源不一致
        }
        var list = getMedList();
        for (var j = 0; j < list.length; j++) {
            if (list[j] && list[j].name === name) {
                list[j].stock = round2((parseFloat(list[j].stock) || 0) + delta);
                persistList(list);
                return list[j].stock;
            }
        }
        return null;
    }
 // ----------------------------------------------------------------
 // 流水账
 // ----------------------------------------------------------------
    var TYPE_LABEL = { 'in': '入库', 'rx': '处方消耗', 'rx-adj': '处方调整', 'rx-revert': '删除冲回', 'adj':
'盘点调整', 'init': '导入建账' };
    function getLedger() {
        try {
            var arr = JSON.parse(localStorage.getItem(LEDGER_KEY) || '[]');
            return Array.isArray(arr) ? arr : [];
        } catch (e) { return []; }
    }
    function saveLedger(arr) {
        try {
            if (!Array.isArray(arr)) return;
            if (arr.length > LEDGER_CAP) arr = arr.slice(0, LEDGER_CAP);
            localStorage.setItem(LEDGER_KEY, JSON.stringify(arr));
        } catch (e) { console.warn('[StockCore] 流水落盘失败:', e); }
    }
    function pushEntry(e) {
        var arr = getLedger();
        arr.unshift(e);
        saveLedger(arr);
    }
    function makeEntry(name, type, qty, stock, note) {
        return {
            t: new Date().toISOString(),
            name: name,
            type: type,
            qty: round2(qty),
            stock: (stock === null || stock === undefined) ? null : round2(stock),
            op: operator(),
            note: note || ''
        };
    }
```

```
// ---------------------------------------------------------------------
// ① 处方扣减引擎（差值冲正 + 幂等标记）
// ---------------------------------------------------------------------
// 纯计算：不产生副作用。返回 {hasChanges, deltas:[{name,delta}], marker}
    function planDeduction(record) {
        var items = (record && Array.isArray(record.items)) ? record.items : [];
        var dose = parseInt(record && record.doseCount, 10) || 0;
        var need = {};
        for (var i = 0; i < items.length; i++) {
            var it = items[i];
            if (!it || !it.name) continue;
            var q = (parseFloat(it.dosage) || 0) * dose;
            if (Math.abs(q) < 0.01) continue;
            need[it.name] = round2((need[it.name] || 0) + q);
        }
        var prev = {};
        try {
            if (record && record.__stockApplied && record.__stockApplied.items &&
                typeof record.__stockApplied.items === 'object') {
                prev = record.__stockApplied.items;
            }
        } catch (e) {}
        var deltas = [];
        var name;
        for (name in need) {
            if (!need.hasOwnProperty(name)) continue;
            var d = round2(need[name] - (parseFloat(prev[name]) || 0));
            if (Math.abs(d) >= 0.01) deltas.push({ name: name, delta: d });
        }
        for (name in prev) { // 处方中已删除的药味：全额退回
            if (!prev.hasOwnProperty(name)) continue;
            if (!(name in need)) {
                var d2 = -round2(parseFloat(prev[name]) || 0);
                if (Math.abs(d2) >= 0.01) deltas.push({ name: name, delta: d2 });
            }
        }
        return {
            hasChanges: deltas.length > 0,
            deltas: deltas,
            marker: { v: 1, at: new Date().toISOString(), items: need }
        };
    }
// 应用扣减计划：改库存 + 记流水，返回预警清单 [{name, stock, threshold}]
    function commitDeduction(plan, rxNo, isFirstApply) {
        var warns = [];
        if (!plan || !plan.hasChanges) return warns;
        for (var i = 0; i < plan.deltas.length; i++) {
            var d = plan.deltas[i];
            var next = applyStockDelta(d.name, -d.delta);
            if (next === null) continue; // 药品库无此药（已删/改名）：跳过
            pushEntry(makeEntry(d.name, isFirstApply ? 'rx' : 'rx-adj', d.delta, next, rxNo ? '处方 ' + rxNo : ''));
            var live = getMedLive(d.name) || getMedList().filter(function (m) { return m.name === d.name; })[0];
            if (live && isLowStock(live)) warns.push({ name: d.name, stock: live.stock, threshold:
getThreshold(live) });
            else if (live && (parseFloat(live.stock) || 0) < 0) warns.push({ name: d.name, stock: live.stock,
```

```
threshold: 0 });
        }
        refreshUI();
        return warns;
    }
 // 彻底删除冲回：有 __stockApplied 标记才冲（防幻影退回）
    function revertRecord(record) {
        var applied = null;
        try { applied = record && record.__stockApplied && record.__stockApplied.items; } catch (e) {}
        if (!applied) return 0;
        var n = 0;
        var rxNo = (record && (record.prescriptionNo || record.outpatientNo)) || '';
        for (var name in applied) {
            if (!applied.hasOwnProperty(name)) continue;
            var qty = parseFloat(applied[name]) || 0;
            if (Math.abs(qty) < 0.01) continue;
            var next = applyStockDelta(name, qty);
            if (next === null) continue;
            pushEntry(makeEntry(name, 'rx-revert', qty, next, rxNo ? '处方 ' + rxNo + ' 删除冲回' : '删除冲回'));
            n++;
        }
        if (n > 0) refreshUI();
        return n;
    }
    function notifyLowStock(warns) {
        if (!warns || !warns.length) return;
        var parts = warns.slice(0, 3).map(function (w) {
            return w.name + '(结存' + w.stock + (w.threshold > 0 ? '/阈值' + w.threshold : '') + ')';
        });
        toast('⚠ 库存预警：' + parts.join('、') + (warns.length > 3 ? ' 等' + warns.length + ' 项' : ''));
    }
 // ---------------------------------------------------------------
 // ② 入库 / 盘点
 // ---------------------------------------------------------------
    function stockIn(name, qty, note) {
        qty = parseFloat(qty) || 0;
        if (!name || Math.abs(qty) < 0.01) return false;
        var next = applyStockDelta(name, qty);
        if (next === null) return false;
        pushEntry(makeEntry(name, 'in', qty, next, note || ''));
        refreshUI();
        return true;
    }
    function recordManualAdjust(name, oldStock, newStock, note) {
        var diff = round2((parseFloat(newStock) || 0) - (parseFloat(oldStock) || 0));
        if (Math.abs(diff) < 0.01) return false;
        pushEntry(makeEntry(name, 'adj', diff, parseFloat(newStock) || 0, note || ''));
        return true;
    }
 // ---------------------------------------------------------------
 // ③ UI 渲染辅助（供 index.html 锚点模板调用）
 // ---------------------------------------------------------------
    function stockText(m) {
        var s = (m && m.stock !== undefined && m.stock !== null) ? m.stock : 0;
        if (!isEnabled()) return String(s);
```

```
        var low = isLowStock(m);
        var neg = (parseFloat(s) || 0) < 0;
        if (low || neg) {
            var t = '<span style="color:#e53935;font-weight:bold;">' + esc(s);
            if (low && getThreshold(m) > 0) t += ' ⚠';
            t += '</span>';
            return t;
        }
        return String(s);
    }
    function currentDoseCount() {
        var ids = ['doseCountInput3', 'doseCountInput', 'doseCountInput3M', 'doseCountInput2'];
        for (var i = 0; i < ids.length; i++) {
            var el = document.getElementById(ids[i]);
            if (el && el.value) { var v = parseInt(el.value, 10); if (v > 0) return v; }
        }
        return 7;
    }
    function dropBadge(m) {
        if (!isEnabled() || !m) return '';
        var s = parseFloat(m.stock) || 0;
        var need = (parseFloat(m.dosage) || 0) * currentDoseCount();
        var insufficient = need > 0 && s > 0 && s < need;
        var low = isLowStock(m);
        if (s <= 0 && !insufficient && !low) return ''; // 0=未维护，不显示（噪音控制）
        var color = (insufficient || low) ? '#e53935' : '#909399';
        return ' <span style="color:' + color + ';font-size:10px;">[库存:' + esc(s) + ']</span>';
    }
/* ========== 源文件：stock-core.js（库存钩子安装：保存 / 删除 / 清空 / 编辑 / 导入 全链路） ========== */
    function tryInstall() {
        if (installed) return true;
        if (typeof global.savePrescriptionToDB !== 'function' ||
            typeof global.saveMedicineEdit !== 'function' ||
            typeof global.showMedicineEditModal !== 'function') {
            return false;
        }
        installed = true;
 // ① 保存处方 → 差值扣减（唯一落库收口：新存/同日覆盖/改历史/回收站恢复全走此）
        var _saveRx = global.savePrescriptionToDB;
        global.savePrescriptionToDB = async function (record) {
            var plan = null, rxNo = '', isFirstApply = false;
            try {
                if (isEnabled()) {
 // 云端同步幂等：D1 不保留顶层 __stockApplied，但 extra JSON 会随处方
 // 持久化/同步；extra 里有标记同样视为“已扣过”，避免同步后编辑时重复扣库存。
                    var __extraMark = (record && record.extra && record.extra.__stockApplied) || '';
                    isFirstApply = !(record && (record.__stockApplied || __extraMark));
                    rxNo = (record && (record.prescriptionNo || record.outpatientNo)) || '';
                    plan = planDeduction(record);
                }
            } catch (e) { plan = null; }
            var result = await _saveRx.apply(this, arguments);
            try {
                if (plan && plan.hasChanges && record && typeof record === 'object') {
                    record.__stockApplied = plan.marker;      // 幂等标记随处方持久化/云端同步
```

```
// 云端 D1 只保留 extra 列：标记同步写一份到 extra，跨端/重装后可恢复，防重复扣减
                    try {
                        record.extra = Object.assign({}, record.extra || {}, { __stockApplied: plan.marker });
                    } catch (e1) {}
                    try { await _saveRx.call(this, record); } catch (e2) {}
                    notifyLowStock(commitDeduction(plan, rxNo, isFirstApply));
                }
            } catch (e) { console.warn('[StockCore] 处方扣库存失败（不影响处方保存）:', e); }
            return result;
        };
// ② 彻底删除/清空回收站 → 冲回（confirm 在原函数内部，须在其成功后判定）
        var _delBin = global.deleteFromRecycleBin;
        if (typeof _delBin === 'function') {
            global.deleteFromRecycleBin = function (id) {
                var rec = null;
                try { if (isEnabled() && typeof global.getRecycleBin === 'function') {
global.getRecycleBin().forEach(function (r) { if (r && r.id === id) rec = r; }); } } catch (e) {}
                var r2 = _delBin.apply(this, arguments);
                try {
                    if (rec && isEnabled() && typeof global.getRecycleBin === 'function' &&
                        !global.getRecycleBin().some(function (x) { return x.id === id; })) {
                        var n = revertRecord(rec);
                        if (n > 0) toast('已冲回 ' + n + ' 味药的处方扣减库存');
                    }
                } catch (e) {}
                return r2;
            };
        }
        var _clearBin = global.clearRecycleBin;
        if (typeof _clearBin === 'function') {
            global.clearRecycleBin = function () {
                var recs = [];
                try { if (isEnabled() && typeof global.getRecycleBin === 'function') { recs =
global.getRecycleBin().filter(function (r) { return r && r.__stockApplied; }); } } catch (e) {}
                var r2 = _clearBin.apply(this, arguments);
                try {
                    if (isEnabled() && recs.length && typeof global.getRecycleBin === 'function' &&
global.getRecycleBin().length === 0) {
                        var n = 0; recs.forEach(function (r) { n += revertRecord(r) || 0; });
                        if (n > 0) toast('已冲回 ' + n + ' 条处方扣减库存');
                    }
                } catch (e) {}
                return r2;
            };
        }
// ③ 药品编辑：阈值字段注入 + 盘点调整自动记账
        var _showEdit = global.showMedicineEditModal;
        global.showMedicineEditModal = function (m, isAdd) {
            var r = _showEdit.apply(this, arguments);
            try { ensureEditThresholdField(m); } catch (e) {}
            return r;
        };
        var _saveMed = global.saveMedicineEdit;
        global.saveMedicineEdit = function () {
            var name = '', oldStock = 0, wasAdd = false, thr = 0;
```

```
            try {
                var el = document.getElementById('medEditName');
                name = (el && el.value ? el.value : '').trim();
                var existing = (typeof global.getMedicineByName === 'function') ? global.getMedicineByName(name) :
null;
                wasAdd = !existing;
                oldStock = existing ? (parseFloat(existing.stock) || 0) : 0;
                var thrEl = document.getElementById('medEditThreshold');
                thr = thrEl ? (parseFloat(thrEl.value) || 0) : 0;
            } catch (e) {}
            var r = _saveMed.apply(this, arguments);
            try {
                if (name) {
                    var saved = (typeof global.getMedicineByName === 'function') ? global.getMedicineByName(name) :
null;
                    if (saved) {
                        var changed = false;
                        if (isEnabled() && (getThreshold(saved) || 0) !== thr) {
                            saved.stockThreshold = thr; changed = true;
                        }
                        if (isEnabled()) {
                            var newStock = parseFloat(saved.stock) || 0;
                            if (!wasAdd && Math.abs(newStock - oldStock) >= 0.01) {
                                recordManualAdjust(name, oldStock, newStock, '编辑药品'); changed = true;
                            } else if (wasAdd && newStock > 0) {
                                pushEntry(makeEntry(name, 'in', newStock, newStock, '新增药品期初库存')); changed =
true;
                            }
                        }
                        if (changed) {
                            persistList(getMedList());
                            if (typeof global.renderMedicineList === 'function') global.renderMedicineList();
                        }
                    }
                }
            } catch (e) { console.warn('[StockCore] 药品编辑后处理失败:', e); }
            updateStockBadges();
            return r;
        };
 // ④ 药品列表渲染后刷新红点（列表模板中的红字由锚点编辑直接渲染）
        var _render = global.renderMedicineList;
        if (typeof _render === 'function') {
            global.renderMedicineList = function () {
                var r = _render.apply(this, arguments);
                updateStockBadges();
                syncMedicineModalUI();
                return r;
            };
        }
 // ⑤ 备份恢复：流水 + 开关随备份走（备份导出字段由 index.html 锚点编辑注入 data 对象）
        var _imp = global.importDataFromJson;
        if (typeof _imp === 'function') {
            global.importDataFromJson = async function (jsonStr) {
                var r = await _imp.apply(this, arguments);
                try { restoreFromBackup(jsonStr); } catch (e) {}
```

```
                return r;
            };
        }
// © 导入药品建账（A 期）：快照差值记账——覆盖模式静默清零库存自此可追溯。
// 快照/差值双采集与三模式内部实现解耦：覆盖取消（confirm 拒绝）时前后无差，零流水。
        var _execImport = global.executeImportMethod;
        if (typeof _execImport === 'function') {
            global.executeImportMethod = function (method) {
                var before = null;
                try { if (isEnabled()) before = snapshotStockMap(); } catch (e0) {}
                var r = _execImport.apply(this, arguments);
                try {
                    if (before && isEnabled()) {
                        var n = recordImportLedger(method, before);
                        if (n > 0) toast('已记「导入建账」流水 ' + n + ' 笔（库存流水可查）');
                    }
                } catch (e) { console.warn('[StockCore] 导入建账失败（不影响导入）:', e); }
                return r;
            };
        }
        injectSettingsToggle();
        injectMedicineModalButtons();
        updateStockBadges();
        return true;
    }
/* ========== 源文件：analytics-core.js（统计口径：初复诊 / 病种分布 / 药材用量 / 就诊趋势） ========== */
        function getFilteredPrescriptions(rangeId) {
            const elementId = rangeId || 'analyticsRange';
            const range = document.getElementById(elementId)?.value || 'month';
            const now = new Date();
            let startDate = new Date(1970, 0, 1);
            switch(range) {
                case 'today':
                    startDate = new Date(now.getFullYear(), now.getMonth(), now.getDate());
                    break;
                case 'week':
                    startDate = new Date(now.getTime() - 7 * 24 * 60 * 60 * 1000);
                    break;
                case 'month':
                    startDate = new Date(now.getFullYear(), now.getMonth(), 1);
                    break;
                case 'quarter':
                    const quarter = Math.floor(now.getMonth() / 3);
                    startDate = new Date(now.getFullYear(), quarter * 3, 1);
                    break;
                case 'year':
                    startDate = new Date(now.getFullYear(), 0, 1);
                    break;
            }
            const sourceData = prescriptionHistory;
            return sourceData.filter(p =>{
                const date = new Date(p.date || p.createdAt || 0);
                return date >= startDate;
            });
        }
```

```
        function analyzePatients() {
            const filteredPrescriptions = getFilteredPrescriptions();
            const patientMap = new Map();
            filteredPrescriptions.forEach(p =>{
                const name = p.patientName || '未知';
                if (!patientMap.has(name)) {
                    patientMap.set(name, {
                        visits: [],
                        firstVisit: null,
                        lastVisit: null
                    });
                }
                const patient = patientMap.get(name);
// 日期优先级反转：p.date（就诊日期，可经 editPrescriptionDate 补录修改）
// 优先于 createdAt（首建时间戳）；云端拉取的记录 createdAt 恒为首建值，
// 若其优先则补录日期在患者分析页不生效（历史列表/统计页均按 p.date 归位）
                const visitDate = new Date(p.date || p.createdAt || 0);
                patient.visits.push(visitDate);
                if (!patient.firstVisit || visitDate< patient.firstVisit) {
                    patient.firstVisit = visitDate;
                }
                if (!patient.lastVisit || visitDate >patient.lastVisit) {
                    patient.lastVisit = visitDate;
                }
            });
            return Array.from(patientMap.entries()).map(([name, data]) =>({
                name,
                visitCount: data.visits.length,
                firstVisit: data.firstVisit,
                lastVisit: data.lastVisit
            }));
        }
        function analyzeDiseases() {
            const filteredPrescriptions = getFilteredPrescriptions();
            const diseaseMap = new Map();
            filteredPrescriptions.forEach(p =>{
                const diagnosis = p.diagnosis || '未诊断';
                diseaseMap.set(diagnosis, (diseaseMap.get(diagnosis) || 0) + 1);
            });
            return Array.from(diseaseMap.entries())
                .map(([name, count]) =>({ name, count }))
                .sort((a, b) =>b.count - a.count);
        }
        function analyzeMedicines() {
            const filteredPrescriptions = getFilteredPrescriptions();
            const medicineMap = new Map();
            filteredPrescriptions.forEach(p =>{
                if (p.items && Array.isArray(p.items)) {
                    p.items.forEach(m =>{
                        const name = m.name || m.medicineName || '未知';
                        const dosage = parseFloat(m.dosage) || 0;
                        const doseCount = parseFloat(p.doseCount) || 1;
                        medicineMap.set(name, (medicineMap.get(name) || 0) + dosage * doseCount);
                    });
                }
```

```
            });
            return Array.from(medicineMap.entries())
                .map(([name, totalDosage]) =>({ name, totalDosage }))
                .sort((a, b) =>b.totalDosage - a.totalDosage)
                .slice(0, 10);
        }
        function formatDate(date) {
            if (!date) return '-';
            return date.toLocaleDateString('zh-CN');
        }
        function analyzeVisitTrend() {
            const filteredPrescriptions = getFilteredPrescriptions();
            const dailyMap = new Map();
            filteredPrescriptions.forEach(p =>{
                const date = new Date(p.date || p.createdAt || 0);
// 键改 ISO YYYY-MM-DD：旧 zh-CN「」按字符串排序会错排
// （9 日排到 10 日之后）；ISO 键字典序即时间序，X 轴还能直接截「日」做短标签
                const dateStr = `${date.getFullYear()}-${String(date.getMonth() + 1).padStart(2,
'0')}-${String(date.getDate()).padStart(2, '0')}`;
                dailyMap.set(dateStr, (dailyMap.get(dateStr) || 0) + 1);
            });
            const dates = Array.from(dailyMap.keys()).sort();
            return {
                dates,
                counts: dates.map(d =>dailyMap.get(d))
            };
        }
        function createChart(canvasId, type, data, retryCount = 0) {
            const canvas = document.getElementById(canvasId);
            if (!canvas) return null;
            const ctx = canvas.getContext('2d');
// 修复「数据统计→刷新数据，图表不断变大 / 图表不显示」：
// 根因一：三个 canvas 无 CSS 宽度约束，width/height 属性会反向决定布局尺寸，
// 旧代码把属性写成 rect.width*dpr → 元素变大 → 下次测量更大（每次刷新按比例膨胀）。
// 根因二（复位方案的反噬）：把属性设成 1×baseH 后若把 height 留空（auto），
// 浏览器按 canvas 内在宽高比 1:200 推导高度 → 高度=宽度×200（实测 98500px），
// 超过 16384 上限后 canvas 直接失效，渲染成“破图”图标（图表不显示）。
// 最终对策：高度一律写成显式 px（baseH，首次调用时按 CSS 计算值捕获，含移动端
// 130px 媒体查询），宽度跟随容器；属性只承载设备像素，绝不参与布局推导。
            if (!(canvas.__baseH > 0)) {
                const ch = parseFloat(window.getComputedStyle(canvas).height);
                if (ch > 0) canvas.__baseH = Math.round(ch); // 只在可见时捕获，避免隐藏态取到 0
            }
            const baseH = canvas.__baseH > 0
                ? canvas.__baseH
                : (parseFloat(canvas.getAttribute('height')) || 200);
            canvas.style.width = '100%';
            canvas.style.height = baseH + 'px';
            canvas.width = 1;
            canvas.height = baseH;
            const rect = canvas.getBoundingClientRect();
            if (rect.width === 0 || rect.height === 0) {
                if (retryCount < 3) {
                    setTimeout(() => createChart(canvasId, type, data, retryCount + 1), 100);
                }
```

```
                return null;
            }
            const width = Math.max(1, Math.round(rect.width));
            const height = Math.max(1, Math.round(rect.height));
 // 防御：设备像素超过 16384 会让 canvas 失效（破图），按比例降采样保住画面
            const MAX_DIM = 16384;
            let dpr = window.devicePixelRatio || 1;
            if (width * dpr > MAX_DIM || height * dpr > MAX_DIM) {
                dpr = Math.min(MAX_DIM / width, MAX_DIM / height) || 1;
            }
 // 锁定 CSS 盒尺寸（宽高都用测量值），再写设备像素属性——两者解耦后布局不再随
 // 属性回写变化；窗口尺寸变化时 refreshAnalytics 会重走本函数重新测量。
            canvas.style.width = width + 'px';
            canvas.style.height = height + 'px';
            canvas.width = Math.max(1, Math.round(width * dpr));
            canvas.height = Math.max(1, Math.round(height * dpr));
 // 设置 width/height 属性会重置上下文状态（含变换矩阵），显式一次性设定缩放，
 // 避免旧 ctx.scale 在未重置时累积放大。
            ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
            ctx.clearRect(0, 0, width, height);
 // 响应式参数：根据屏幕宽度调整字体和 padding，提升手机端可读性
            const isMobile = window.innerWidth < 768;
            const isSmall = window.innerWidth < 480;
            const labelFont = isMobile ? '12px Arial' : '10px Arial';
            const valueFont = isMobile ? 'bold 12px Arial' : 'bold 10px Arial';
            const axisFont = isMobile ? '11px Arial' : '10px Arial';
            const labelMaxLen = isMobile ? (isSmall ? 3 : 4) : 5;
            const pieLegendFont = isMobile ? '12px Arial' : '11px Arial';
            const pieLegendMaxLen = isMobile ? (isSmall ? 5 : 7) : 7;
            const hBarFont = isMobile ? '11px Arial' : '11px Arial';
            const hBarLabelMaxLen = isMobile ? (isSmall ? 3 : 4) : 7;
            if (type === 'bar') {
                if (!data.labels || !data.values || data.labels.length === 0) {
                    ctx.fillStyle = '#999';
                    ctx.font = '14px Arial';
                    ctx.textAlign = 'center';
                    ctx.fillText('暂无数据', width / 2, height / 2);
                    return;
                }
                const padding = isMobile
                    ? { top: 18, right: 10, bottom: 32, left: 32 }
                    : { top: 20, right: 20, bottom: 40, left: 50 };
                const chartWidth = width - padding.left - padding.right;
                const chartHeight = height - padding.top - padding.bottom;
/* ========== 源文件：analytics-core.js（月度统计报表：权限隔离与诊费口径自识别） ========== */
        async function analyzeMonthlyStats() {
            const statsMap = new Map();
 // 权限隔离：月度统计必须复用权限过滤，与历史列表口径一致。
 // 修复：此前 getAllUserPrescriptions 不带参数读取全量数据，同一设备
 // 多账户登入时（或云端断网回退本地缓存时）统计会串号泄露其他用户处方。
 // 普通用户仅统计本人（createdBy === username），管理员统计全部。
            const allPrescriptions = filterPrescriptionsByPermission(await getAllUserPrescriptions());
            allPrescriptions.forEach(p => {
                const date = new Date(p.date || p.createdAt || 0);
                const monthKey = `${date.getFullYear()}-${String(date.getMonth() + 1).padStart(2, '0')}`;
```

```
                if (!statsMap.has(monthKey)) {
                    statsMap.set(monthKey, {
                        visits: 0,
                        totalIncome: 0,
                        totalRegFee: 0,
                        totalCost: 0,
                        totalProfit: 0,
                        totalDoses: 0
                    });
                }
                const stats = statsMap.get(monthKey);
                stats.visits++;
 // 修复总收入双重计算诊费：现行全部端（云网页/云桌面/离线 APP/
 // 离线桌面/鸿蒙，2026-07 起）保存时 totalAmount = 药费(单价×用量×剂数)+诊疗费，
 // 此处再加 registrationFee 会把诊费计两遍（云端 D1 回填诊费字段后总收入虚高）。
 // 为兼容极个别旧版/备份导入的 totalAmount 不含诊费记录，用记录内冻结的 item.price
 // 重算药费逐张自识别：≈药费+诊费→已含只取 totalAmount；≈仅药费→旧口径补诊费；
 // 两者都不符（改价/缺价等）默认取 totalAmount，fail-safe 不误加。
                const ta = parseFloat(p.totalAmount) || 0;
                const fee = parseFloat(p.registrationFee) || 0;
                const recDoses = parseFloat(p.doseCount) || 1;
                let medSum = 0;
                if (Array.isArray(p.items)) {
                    p.items.forEach(it => {
                        medSum += (parseFloat(it.price) || 0) * (parseFloat(it.dosage) || 0) * recDoses;
                    });
                }
                stats.totalIncome += (fee > 0 && Math.abs(ta - medSum) < 0.03 && Math.abs(ta - (medSum + fee)) >=
0.03)
                    ? (ta + fee)
                    : ta;
                stats.totalRegFee += fee;
                stats.totalDoses += parseFloat(p.doseCount) || 0;
                if (p.items && Array.isArray(p.items)) {
                    p.items.forEach(item => {
                        const medicine = medicines.find(m => m.name === (item.name || item.medicineName));
                        const costPrice = parseFloat(medicine?.costPrice) || parseFloat(item.costPrice) ||
parseFloat(item.price) || 0;
                        const dosage = parseFloat(item.dosage) || 0;
                        const doseCount = parseFloat(p.doseCount) || 1;
                        stats.totalCost += costPrice * dosage * doseCount;
                    });
                }
            });
            statsMap.forEach((stats, key) => {
                stats.totalProfit = stats.totalIncome - stats.totalCost;
                stats.month = key.split('-')[1];
            });
            return Array.from(statsMap.values())
                .sort((a, b) => b.month - a.month);
        }
/* ========== 源文件：analytics-core.js（统计渲染与图表编排） ========== */
        async function refreshAnalytics() {
          try {
            const patients = analyzePatients();
```

```
            const visitCountFilter = document.getElementById('visitCountFilter')?.value || 'all';
            const sortOption = document.getElementById('analyticsSort')?.value || 'visitCount-desc';
            let filteredPatients = patients;
            if (visitCountFilter !== 'all') {
                const count = parseInt(visitCountFilter);
                if (count === 4) {
                    filteredPatients = patients.filter(p =>p.visitCount >= 4);
                } else {
                    filteredPatients = patients.filter(p =>p.visitCount === count);
                }
            }
            filteredPatients.sort((a, b) =>{
                const [field, dir] = sortOption.split('-');
                let comparison = 0;
                if (field === 'visitCount') {
                    comparison = a.visitCount - b.visitCount;
                } else if (field === 'lastVisit') {
                    comparison = (a.lastVisit?.getTime() || 0) - (b.lastVisit?.getTime() || 0);
                }
                return dir === 'desc' ? -comparison : comparison;
            });
            const totalPatients = patients.length;
            const newPatients = patients.filter(p =>p.visitCount === 1).length;
            const returnPatients = patients.filter(p =>p.visitCount >1).length;
            const returnRate = totalPatients >0 ? ((returnPatients / totalPatients) * 100).toFixed(1) : '0.0';
            document.getElementById('statTotalPatients').textContent = totalPatients;
            document.getElementById('statNewPatients').textContent = newPatients;
            document.getElementById('statReturnPatients').textContent = returnPatients;
            document.getElementById('statReturnRate').textContent = returnRate + '%';
            const trend = analyzeVisitTrend();
            createChart('visitTrendChart', 'bar', {
// X 轴只显示「日」（14 个完整日期 挤叠看不清）；
// ISO 键 YYYY-MM-DD 截第 9 位起并去前导零，跨月处 28/29/30/1/2 连续可辨
                labels: trend.dates.slice(-14).map(d => String(parseInt(d.slice(8), 10))),
                values: trend.counts.slice(-14)
            });
            const diseases = analyzeDiseases().slice(0, 6);
            createChart('diseaseChart', 'pie', {
                labels: diseases.map(d =>d.name),
                values: diseases.map(d =>d.count)
            });
            const medicines = analyzeMedicines();
            createChart('medicineChart', 'horizontalBar', {
                labels: medicines.map(m =>m.name),
                values: medicines.map(m =>m.totalDosage)
            });
            const tbody = document.getElementById('patientVisitList');
            if (filteredPatients.length >0) {
                tbody.innerHTML = filteredPatients.slice(0, 50).map(p =>`<tr style="hover:background:#f5f5f5;"><td
style="border:1px solid #ddd;padding:6px;">${escapeHtml(p.name)}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:center;">${p.visitCount}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:center;">${formatDate(p.firstVisit)}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:center;">${formatDate(p.lastVisit)}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:center;"><button class="small-btn"
  onclick="loadPatientHistory('${escapeJs(p.name)}')" style="padding:2px
```

```
  8px;">调取处方</button></td></tr>`).join('');
            } else {
                tbody.innerHTML = '<tr><td colspan="5"
style="text-align:center;padding:20px;color:#999;">暂无数据</td></tr>';
            }
            const monthlyStats = await analyzeMonthlyStats();
            const statsTbody = document.getElementById('monthlyStatsTable');
            if (monthlyStats.length > 0) {
                statsTbody.innerHTML = monthlyStats.map(s =>`<tr style="hover:background:#f5f5f5;"><td
style="border:1px solid #ddd;padding:6px;text-align:center;">${escapeHtml(s.month)}月</td><td style="border:1px
  solid #ddd;padding:6px;text-align:center;">${s.visits}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:right;">${formatPrice(s.totalIncome)}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:right;">${formatPrice(s.totalRegFee)}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:right;">${formatPrice(s.totalCost)}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:right;">${formatPrice(s.totalProfit)}</td><td style="border:1px solid
  #ddd;padding:6px;text-align:center;">${s.totalDoses}</td></tr>`).join('');
            } else {
                statsTbody.innerHTML = '<tr><td colspan="7"
style="text-align:center;padding:20px;color:#999;">暂无数据</td></tr>';
            }
          } catch (e) {
            console.error('refreshAnalytics 执行失败:', e);
            alert('数据统计加载失败: ' + e.message);
          }
        }
/* ========== 源文件：analytics-core.js（报表导出与付费墙口径） ========== */
 // 窗口尺寸变化时防抖重绘图表（适配手机横竖屏切换）
        let _analyticsResizeTimer = null;
        let _analyticsLastWidth = window.innerWidth;
        window.addEventListener('resize', () => {
            const currentWidth = window.innerWidth;
            if (currentWidth === _analyticsLastWidth) return;
            _analyticsLastWidth = currentWidth;
            if (_analyticsResizeTimer) clearTimeout(_analyticsResizeTimer);
            _analyticsResizeTimer = setTimeout(() => {
                const analyticsModal = document.getElementById('analyticsModal');
                if (analyticsModal && analyticsModal.style.display === 'flex') {
                    refreshAnalytics();
                }
            }, 300);
        });
        function loadPatientHistory(patientName) {
            closeModal('analyticsModal');
            const nameInput = document.getElementById('patientName');
            nameInput.value = patientName;
 // 修复：必须带 target（searchPatientsQuick 读 event.target.value）。
 // 旧写法只传 {key:'Enter'} → input 落 '' → 防抖后搜空串覆盖侧栏为"请输入患者姓名"。
            searchPatientsQuick({ target: nameInput, key: 'Enter' });
            showSideHistory();
        }
        async function exportStats() {
 // 免费版付费墙：统计报表导出（data-export；屏幕内统计查看仍免费）。
 // 仅离线端存在 requireFeature；云端网页无此函数自动跳过。
/* ========== 源文件：permission.js（版本归一化与单一读取链） ========== */
 // 根治【版本状态双轨制】：edition 动态权威判定（单一读取链）
```

```
// 旧实现：_edition 在 init 时锁死；登录机构版后 CONFIG.edition / window.EDITION 已被
// getAppConfig/refreshVersionTags 更新为机构版，但 _edition 仍是初始 cloud_personal →
// isInstitutional=false → 机构版管理员看不到【用户管理】。
// 修复：每次判定实时读取 CONFIG.edition → window.EDITION → this._edition，
// 任何一处更新（登录/激活/配置同步）立即生效，标签与权限永远同轨。
// Arch 2.25 根治【edition 别名漏判——云端机构版按钮复发真根因】：
// 激活流程（activate-schema.js/auth-core.js/activate-window.html）写入 userData 的
// edition 值是 'institution'（机构）/ 'standard'（标准）/ 中文标签（云端机构版…），
// 而 enforceEditionBinding 只在有"验签通过的正式 license"时才纠正为规范 key。
// 无 license 的云端激活 → userData config.json 永远是 'institution' →
// isInstitutional 精确列表不命中 → canManageUsersByRole=false、
// canChangePassword=true → 管理员看到【修改密码】而非【用户管理】，
// 同时 refreshVersionTags 的列表含 'institution' → 标签却显示【云端机构版】。
// 修复：_currentEdition 统一归一化（单一读取链），所有谓词自动拿到规范 key。
      _normalizeEdition(e) {
          var s = String(e || '').trim();
          if (!s) return s;
          var x = s.toLowerCase();
          if (x === 'institution' || x === 'institutional' || x === 'jigou') return 'cloud_clinic';
          if (x === 'standard') return 'personal';
// 语音版：cloud_voice 规范 key 原样透传（独立版本线，
// 不归并 personal/clinic；与 edition-lock.js __normalizeEdition 逐行同步）
          if (x === 'cloud_voice' || x === 'voice') return 'cloud_voice';
          if (x === 'yj') return 'cloud_clinic';
          if (x === 'yb') return 'cloud_personal';
          if (x === 'lj') return 'offline_clinic';
          if (x === 'lb') return 'offline_personal';
          if (x.indexOf('云端机构') >= 0) return 'cloud_clinic';
          if (x.indexOf('云端标准') >= 0) return 'cloud_personal';
          if (x.indexOf('离线机构') >= 0) return 'offline_clinic';
          if (x.indexOf('离线标准') >= 0) return 'offline_personal';
          if (x.indexOf('机构版') >= 0) return 'clinic';
          if (x.indexOf('标准版') >= 0) return 'personal';
          return s;
      },
// 【热更环境机构版按钮消失根治】读取 userData 权威 edition 插槽。
// 事故链：cloud 热通道长期错投本地身份 index.html（EDITION=personal/惠康中医-本地/
// APP_MODE=auto），热目录又无 config.json（主进程复制的是 asar 出厂模板 cloud_personal）
// → CONFIG.edition 与 window.EDITION 全是 personal 系；唯一的真相是 Permission.init
// 经 IPC 从 userData 读到并暂存的权威值（9-01 已写插槽，但按钮判定链从不读它）。
// 规则：插槽值（userData 激活配置）> CONFIG.edition（同目录 XHR，热目录可能是出厂模板）
// > window.EDITION（出厂默认，必然不可信）> this._edition。
// 注意 TDZ：CONFIG 在解析期可能处于暂时性死区，typeof 亦抛错——必须 try-catch。
      _authoritativeEditionValue() {
          try {
              if (typeof CONFIG !== 'undefined' && CONFIG && CONFIG.__authoritativeEdition) {
                  return String(CONFIG.__authoritativeEdition);
              }
          } catch (_) {}
          try {
              if (this._authoritativeEdition) return String(this._authoritativeEdition);
          } catch (_) {}
          return '';
      },
      _currentEdition() {
```

```
 // 权威插槽最优先（热更环境 XHR/默认值均不可信，详见上方法注释）
            var auth = this._authoritativeEditionValue();
            if (auth) return this._normalizeEdition(auth);
            var v = '';
            try {
                if (typeof CONFIG !== 'undefined' && CONFIG && CONFIG.edition) v = String(CONFIG.edition);
            } catch (_) {}
            try {
                if (!v && global.EDITION) v = String(global.EDITION);
            } catch (_) {}
            if (!v) v = String(this._edition || '');
            return this._normalizeEdition(v);
        },
 // 主动同步 edition（登录后版本切换时可调用；三处同写保持单一权威）
        setEdition(ed) {
            var v = String(ed || '').trim();
            if (!v) return;
            this._edition = v;
            try { if (this._config) this._config.edition = v; } catch (_) {}
            try { if (typeof CONFIG !== 'undefined' && CONFIG) CONFIG.edition = v; } catch (_) {}
            try { global.EDITION = v; } catch (_) {}
        },
/* ========== 源文件：permission.js（角色权限矩阵：用户管理 / 改密 / 全所处方查阅） ========== */
 // 是否可以管理用户（仅机构版可管理子账号；标准版/单用户一律不可）
        canManageUsersByRole(user) {
 // 标准版强制守护：一律不可管理用户（隐藏用户管理按钮）
            if (this._isStandardEditionForced()) return false;
            if (!this.isInstitutional()) return false;
            if (!user) return false;
            if (global.AuthCore && global.AuthCore.isClinicAdmin) {
                return global.AuthCore.isClinicAdmin(user);
            }
 // 回退：直接角色比较
            return user.role === 'admin' || user.role === 'clinic_admin';
        },
 // 是否可以修改密码（准则：试用=标准版=单用户=修改密码）
 // 非机构版（标准版/单用户）所有账号均可修改密码；
 // 机构版仅普通用户可修改密码，管理员使用账户管理（不显示修改密码）。
        canChangePassword(user) {
 // 标准版强制守护：所有角色一律允许修改密码（显示改密按钮）
            if (this._isStandardEditionForced()) return true;
            if (!this.isInstitutional()) return true; // 标准版/单用户允许改密
            if (!user) return false;
 // 非个人版：管理员不显示修改密码（由账户管理覆盖），普通用户显示修改密码
            if (global.AuthCore && global.AuthCore.isClinicAdmin) {
                return !global.AuthCore.isClinicAdmin(user);
            }
            return user.role !== 'admin' && user.role !== 'clinic_admin';
        },
 // 是否显示账户管理按钮
        shouldShowUserManage(user) {
            return this.canManageUsersByRole(user);
        },
 // 是否显示修改密码按钮
        shouldShowChangePwd(user) {
```

```
            return this.canChangePassword(user);
        },
// 是否显示开机自启选项
        shouldShowAutoStart(user) {
            if (this.isPersonal()) return false;
            if (!user) return false;
            if (global.AuthCore && global.AuthCore.isClinicAdmin) {
                return global.AuthCore.isClinicAdmin(user);
            }
            return user.role === 'admin' || user.role === 'clinic_admin';
        },
// 是否可以查看所有处方（管理员可查看全部，普通用户只能查看自己的）
// 前台收费：cashier 与管理员一样可读全所处方（收费工作台数据源）
        canViewAllPrescriptions(user) {
// 标准版强制守护：单用户版本不显示处方查阅/收费入口
            if (this._isStandardEditionForced()) return false;
            if (!user) return false;
            if (global.AuthCore && global.AuthCore.isClinicAdmin) {
                return global.AuthCore.isClinicAdmin(user) || global.AuthCore.isCashier(user);
            }
            return user.role === 'admin' || user.role === 'clinic_admin' || user.role === 'cashier';
        },
// 是否可以执行收费动作（管理员 + 前台收费）
        canChargePrescriptions(user) {
            return this.canViewAllPrescriptions(user);
        },
// 应用运行页权限控制
        applyRuntimePermissions() {
            const edition = this._currentEdition();
            console.log('[DBG] Applying runtime permissions for edition:', edition);
// 诊所名称字段（ 新规范：所有版本允许修改诊所名称）
// 旧规则：personal/clinic_custom 设为只读，已废弃
// const clinicNameInput = document.getElementById('clinicName');
// if (clinicNameInput) {
// if (edition === 'personal' || edition === 'clinic_custom') {
// clinicNameInput.readOnly = true;
// clinicNameInput.style.backgroundColor = '#f0f0f0';
// clinicNameInput.style.cursor = 'not-allowed';
// clinicNameInput.title = '当前版本不支持修改诊所名称';
// }
// }
// 医师姓名字段（ 新规范：所有版本允许修改医师姓名）
// 旧规则：personal 设为只读，已废弃
// if (edition === 'personal') {
// const defaultDoctorInput = document.getElementById('defaultDoctor');
// const doctorNameInput = document.getElementById('doctorName');
// [defaultDoctorInput, doctorNameInput].forEach(el => {
// if (el) {
// el.readOnly = true;
// el.style.backgroundColor = '#f0f0f0';
// el.style.cursor = 'not-allowed';
// el.title = '当前版本不支持修改医师姓名';
// }
// });
// }
```

```
/* ========== 源文件：voice-input.js（语音录入三重门控 / 连报守护 / 多候选） ========== */
 // 三重条件入口判定（唯一 gate，注入按钮前必须调用）
    VoiceInput.isAvailable = function () {
        return !!(getRecognitionCtor() && isSecureCtx() && hasVoiceEntitlement());
    };
 // ── 2. ASR 识别（一期：Web Speech API） ──────────────────────
    var listening = false;
    var recognition = null;
    var stopRequested = false;   // 主动停止标志（VoiceInput.stop / 静音守护置位）
    var lastActivity = 0;        // 最近识别活动时刻（连报守护重启判据）
    var watchdogTimer = null;    // 连报静音守护计时器（8s 无语音自动收尾）
    var sessionSeq = 0;          // 识别会话序号（防旧会话异步回调错误重启/收尾新会话）
 // 防重入：识别中重复点击直接忽略
    VoiceInput.isListening = function () { return listening; };
 // 提速：主动停止（连报模式再点 🎤 / 医生想立即中断时调用）。
 // abort → onend（stopRequested 已置位不触发守护重启）→ 正常收尾调 onEnd。
    VoiceInput.stop = function () {
        stopRequested = true;
        if (watchdogTimer) { clearInterval(watchdogTimer); watchdogTimer = null; }
        try { if (recognition) recognition.abort(); } catch (e) {}
    };
 /**
 * 单次/连报语音识别（按下 🎤 → 说话 → 自动结束回调）
 * @param {Function} onResult (text:string, alts:string[]) => void 最终结果回调（已 trim）
 * @param {Function} onError (msg:string) => void 失败/拒绝回调（用户可读中文）
 * @param {Function} onEnd => void 识别结束回调（无论成败，用于恢复按钮态）
 * @param {Object} [opts] 提速增强（全部可选，旧调用方零影响）：
 * continuous : true 连报模式——说完一句不停继续听（连报多味药免反复点🎤）；
 * Chrome 静音数秒会静默 onend，6s 内有活动自动守护重启续听；
 * 8 秒无语音自动收尾；VoiceInput.stop 立即停。
 * onInterim : fn(text) 中间结果实时回调（说话期间字随话出，仅供字幕展示，
 * 绝不触发填充——final 结果才走 onResult 业务链）。
 */
    VoiceInput.listen = function (onResult, onError, onEnd, opts) {
        var Ctor = getRecognitionCtor();
        if (!Ctor) {
            if (typeof onError === 'function') onError('当前环境不支持语音识别');
            if (typeof onEnd === 'function') onEnd();
            return;
        }
        if (listening) {
            try { if (recognition) recognition.abort(); } catch (_) {}
            listening = false;
        }
        try {
            recognition = new Ctor();
        } catch (e) {
            if (typeof onError === 'function') onError('语音识别初始化失败');
            if (typeof onEnd === 'function') onEnd();
            return;
        }
        var continuous = !!(opts && opts.continuous);
        var onInterim = (opts && typeof opts.onInterim === 'function') ? opts.onInterim : null;
        recognition.lang = 'zh-CN';
 // 提速：interimResults=true——说话期间实时吐中间结果
```

```
// （onInterim 字幕展示，"字随话出"确定感大增）；final 行为不变。
        recognition.interimResults = true;
// 连报修复：多候选——ASR 对药名常输出同音字（白芍→白勺），
// 收集全部候选供业务层逐个尝试匹配（单候选时同音字必 miss）。
        recognition.maxAlternatives = 5;
// 提速：continuous 连报模式（药物 mic 用）——说完一句不停
// 继续听，免反复点🎤；患者信息各框保持单句模式说到自动结束。
        recognition.continuous = continuous;
        listening = true;
        var mySeq = ++sessionSeq;   // 本会话序号：旧会话的异步回调据此失效
        stopRequested = false;      // 新一轮识别：清除上一轮的主动停止标志
        lastActivity = Date.now();
        if (watchdogTimer) { clearInterval(watchdogTimer); watchdogTimer = null; }
        recognition.onresult = function (event) {
            lastActivity = Date.now();
            try {
// 只看最后一行（单句=唯一行；连报=最新 utterance）
                var r = (event && event.results && event.results.length > 0)
                    ? event.results[event.results.length - 1] : null;
                if (!r || r.length === 0) return;
// 提速：中间结果只做实时字幕，绝不填充（final 才走业务链）
                if (!r.isFinal) {
                    if (onInterim) {
                        var it = String(r[0].transcript || '').trim();
                        if (it) onInterim(it);
                    }
                    return;
                }
                var text = '';
                var alts = [];
                for (var ai = 0; ai < r.length; ai++) {
                    var at = String(r[ai].transcript || '').trim();
                    if (!at) continue;
                    if (!text) text = at;
                    if (alts.indexOf(at) < 0) alts.push(at);
                }
// onResult(text, alts)：alts = 全部候选文本（含首选，去重），旧调用方只收 text 不受影响
                if (text && typeof onResult === 'function') onResult(text, alts);
                else if (!text && typeof onError === 'function' && !continuous) onError('未识别到内容，请再试一次');
            } catch (e) {
                if (typeof onError === 'function') onError('识别结果处理异常');
            }
        };
        recognition.onerror = function (event) {
            var code = (event && event.error) || '';
            var msg;
            if (code === 'not-allowed' || code === 'service-not-allowed') {
                msg = '麦克风权限被拒绝，请在浏览器地址栏允许麦克风后重试';
            } else if (code === 'no-speech') {
// 连报模式静音间隙的 no-speech 静默（onend 守护重启续听）；
// 单句模式才提示医生
                msg = continuous ? '' : '未检测到语音，请靠近麦克风重试';
            } else if (code === 'network') {
                msg = '语音服务网络异常，请检查网络后重试';
            } else if (code === 'aborted') {
```

```
                msg = '';  // 主动中止不算错误（静默）
            } else {
                msg = '语音识别失败（' + code + '），请重试';
            }
            if (msg && typeof onError === 'function') onError(msg);
        };
        function finish() {
            if (mySeq !== sessionSeq) return;   // 旧会话迟到回调：新会话已接管，全丢弃
            listening = false;
            if (watchdogTimer) { clearInterval(watchdogTimer); watchdogTimer = null; }
            if (typeof onEnd === 'function') onEnd();
        }
// 提速连报：Chrome continuous 模式静音数秒会静默 onend（引擎行为，
// 不可配置）——6 秒内有识别活动则 200ms 后守护重启续听，医生无感；
// 超窗 / 主动停止则正常收尾。
        recognition.onend = function () {
            if (mySeq !== sessionSeq) return;   // 旧会话迟到回调：不影响新会话
            if (continuous && !stopRequested && listening &&
                (Date.now() - lastActivity) < 6000) {
                setTimeout(function () {
                    if (mySeq !== sessionSeq || stopRequested || !listening) return;
                    try {
                        recognition.start();
                        lastActivity = Date.now();
                        return;
                    } catch (e) {}
                    finish();
                }, 200);
                return;
            }
            finish();
        };
// 提速连报：8 秒无任何语音自动收尾（防忘关麦克风一直收诊室环境音）
        if (continuous) {
            watchdogTimer = setInterval(function () {
                if (stopRequested || !listening) {
                    if (watchdogTimer) { clearInterval(watchdogTimer); watchdogTimer = null; }
                    return;
                }
                if ((Date.now() - lastActivity) >= 8000) {
                    stopRequested = true;   // 阻断 onend 守护重启，走正常收尾
                    try { recognition.abort(); } catch (e) {}
                }
            }, 1500);
        }
        try {
            recognition.start();
        } catch (e) {
// start 抛异常（如已启动竞态）——按失败处理
            listening = false;
            if (watchdogTimer) { clearInterval(watchdogTimer); watchdogTimer = null; }
            if (typeof onError === 'function') onError('语音识别启动失败，请重试');
            if (typeof onEnd === 'function') onEnd();
        }
    };
```

```
 // ── 3. 高亮辅助（语音填充内容必须高亮待确认） ──────────
 // 浅紫色高亮：target 填充后加 class + 2.5s 后淡出（医生可继续编辑）
    var HIGHLIGHT_CLASS = 'voice-filled-highlight';
    var highlightStyleInjected = false;
    function ensureHighlightStyle() {
        try {
            if (highlightStyleInjected) return;
            if (typeof document === 'undefined' || !document.head) return;
            var style = document.createElement('style');
            style.id = 'voice-input-highlight-style';
            style.textContent = '.' + HIGHLIGHT_CLASS + '{background:#f3eefa !important;box-shadow:0 0 0 1px
#b9a6f2 inset !important;transition:background .6s,box-shadow .6s;}';
            document.head.appendChild(style);
            highlightStyleInjected = true;
        } catch (_) {}
    }
    VoiceInput.highlight = function (el) {
        try {
            if (!el || !el.classList) return;
            ensureHighlightStyle();
            el.classList.add(HIGHLIGHT_CLASS);
 // 2.5s 后淡出高亮（内容保留，仅提示色消退；再次填充会重新高亮）
            setTimeout(function () {
                try { el.classList.remove(HIGHLIGHT_CLASS); } catch (_) {}
            }, 2500);
        } catch (_) {}
    };
 // ── 3.5 拼音容错（ 连报修复：ASR 药名同音字根治） ──
/* ========== 源文件：security-guard.js（反调试强/弱信号 + 延迟降级） ========== */
 // ==================== 反调试检测（轻量级） ====================
 // 强信号：debugger 语句时间差（DevTools/调试器附加时必现暂停，可靠）
 // 弱信号：DevTools 尺寸差（缩放/WebView 误报率高，仅记录日志）
        _startAntiDebug() {
            const self = this;
 // 方法 1：时间差检测（每 5 秒检测一次，唯一可触发降级的信号）
 // 当调试器暂停执行时，时间差会显著变大
            setInterval(() => {
                if (!self._enabled) return;
                const start = Date.now();
 // debugger 语句在调试器打开时会暂停执行
 // 使用 constructor 间接调用，避免静态分析直接检测 debugger 关键字
                try {
                    (function () { /* constructor */ }).constructor('debugger').call();
                } catch (e) { /* 某些环境会抛错，忽略 */ }
                const elapsed = Date.now() - start;
 // 正常情况下 elapsed < 10ms，调试器打开时会显著变大
                if (elapsed > 100) {
                    self._strongStrikes++;
                    self._lastSignalTime = Date.now();
 // 连续多次强信号才确认（防单次 GC 抖动误报）
                    if (self._strongStrikes >= CONFIRM_STRIKES && !self._debuggerDetected) {
                        self._debuggerDetected = true;
                        self._debuggerDetectionTime = Date.now();
                        console.warn('[SecurityGuard] 检测到调试器附加（时间差: ' + elapsed + 'ms）');
                    }
```

```
                } else {
                    self._strongStrikes = 0;
// 自愈：强信号消失超过恢复窗口 → 复位（防持续误报困死正常用户）
                    if (self._debuggerDetected && self._lastSignalTime &&
                        Date.now() - self._lastSignalTime > RECOVERY_MS) {
                        self._debuggerDetected = false;
                        self._debuggerDetectionTime = null;
                        self._strongStrikes = 0;
                    }
                }
            }, 5000);
// 方法 2：检测 DevTools 是否打开（仅 Chromium 内核）
// 通过 window 尺寸差判断
// ⚠ 仅记录日志，永不参与降级决策：
// 浏览器缩放（outerWidth 不随 zoom 变化、innerWidth 随之变化）及
// 部分 Android WebView 的 outer/inner 天然差值 >160px，误报率高
            setInterval(() => {
                if (!self._enabled || self._devToolsSuspected) return;
                try {
                    const threshold = 160;
                    const widthDiff = window.outerWidth - window.innerWidth;
                    const heightDiff = window.outerHeight - window.innerHeight;
// DevTools 打开时会有显著尺寸差（>threshold）
                    if (widthDiff > threshold || heightDiff > threshold) {
                        self._devToolsSuspected = true;
                        console.warn('[SecurityGuard] 检测到 DevTools 可能已打开（弱信号，仅记录）');
                    }
                } catch (e) { /* 忽略 */ }
            }, 10000);
        },
// ==================== 延迟静默降级 ====================
// 持续调试 ≥5min（一级）：关键业务操作随机延迟 0.8~2.2s，功能可用但明显变慢
// 持续调试 ≥10min（二级）：关键业务操作拒绝执行，通用话术提示（内部高频调用静默拒绝）
// 强信号消失 90s 自动复位，正常用户零感知
        _installBusinessGuards() {
            const self = this;
// 包装单个全局业务函数；返回 false 表示函数尚未定义（等待重试）
            const wrapTarget = (name, silent) => {
                try {
                    const orig = global[name];
                    if (typeof orig !== 'function') return false;
                    if (orig.__sgWrapped) return true;
                    const wrapped = function (...args) {
                        const level = self.getDegradeLevel();
                        if (level >= 2) {
// 二级降级：拒绝执行
// savePrescription/handleLogin 正常早退模式即 "提示 + return undefined"，
// 此处返回 Promise<undefined> 与原契约完全兼容
                            if (!silent) self._notifyBlocked();
                            return Promise.resolve(undefined);
                        }
                        if (level === 1) {
// 一级降级：随机延迟后正常执行（返回值透传）
                            const delay = L1_DELAY_MIN + Math.floor(Math.random() * (L1_DELAY_MAX - L1_DELAY_MIN));
                            return new Promise(resolve => {
```

```
                            setTimeout(() => resolve(orig.apply(this, args)), delay);
                        });
                    }
                    return orig.apply(this, args);
                };
                wrapped.__sgWrapped = true;
                global[name] = wrapped;
                return true;
            } catch (e) {
                return false;
            }
        };
        const install = () => {
            if (self._guardsInstalled) return;
            let allOk = true;
// 用户触发的关键操作（拒绝时给通用提示）
            if (!wrapTarget('savePrescription', false)) allOk = false;
            if (!wrapTarget('handleLogin', false)) allOk = false;
// 内部数据加载（拒绝时静默，避免高频弹窗）
            if (!wrapTarget('loadData', true)) allOk = false;
            if (allOk) self._guardsInstalled = true;
        };
// 主脚本（含业务函数定义）在 DOMContentLoaded 前执行完毕；
// 延迟安装 + 重试兜底（脚本异常时函数可能尚未定义）
        let retries = 0;
        const tick = () => {
            install();
            if (!self._guardsInstalled && retries < 10) {
                retries++;
                setTimeout(tick, 3000);
            }
        };
        if (document.readyState === 'complete' || document.readyState === 'interactive') {
            setTimeout(tick, 1500);
        } else {
            document.addEventListener('DOMContentLoaded', () => setTimeout(tick, 1500));
        }
    },
// 二级降级提示：通用业务话术 + 节流（不暴露安全检测逻辑）
    _notifyBlocked() {
        const now = Date.now();
        if (now - this._lastBlockAlertTime < BLOCK_ALERT_THROTTLE) return;
        this._lastBlockAlertTime = now;
        try { alert('操作超时，请稍后重试'); } catch (e) { /* 忽略 */ }
    },
// ==================== 完整性校验 ====================
// 策略：校验关键 DOM 元素是否存在，防止 HTML 被篡改
// 不做 hash 校验（计算开销大，且每次版本更新 hash 都会变）
    _checkIntegrity() {
        if (!this._enabled || this._integrityChecked) return;
        try {
// 校验关键 DOM 元素是否存在
            const criticalElements = [
                'prescriptionPaper',   // 处方签纸
                'prescriptionBody',    // 药物表体
```

```
                    'historyList',          // 历史列表
                    'clinicNameDisplay'     // 诊所名显示
                ];
                const missing = criticalElements.filter(id => !document.getElementById(id));
                if (missing.length > 0) {
                    console.warn('[SecurityGuard] 完整性校验失败，缺失关键元素:', missing);
 // 仅记录日志，不强制退出
                }
 // 校验关键全局函数是否存在（防止 JS 被篡改后丢失关键函数）
                const criticalFunctions = [
                    'savePrescription',
                    'loadData',
                    'handleLogin'
                ];
/* ========== 源文件：edition-lock.js（版本配置拦截与三写同步） ========== */
    function tryInstallLock() {
        if (typeof CONFIG === 'undefined') return false;
        if (CONFIG.__editionLocked) return true; // 已安装
        var _slot = CONFIG.edition || '';
 // 安装锁的同时立即归一化现有槽值（否则旧 config.json 中的 institution 不会被纠正）
        _slot = __normalizeEdition(_slot);
 // 重新定义 CONFIG.edition 属性（CONFIG 是 const 变量，但其属性描述符可以改）
        try {
            Object.defineProperty(CONFIG, 'edition', {
                configurable: true,
                enumerable: true,
                get: function () {
                    var v = '';
 // 1. 优先返回权威插槽（由 electronAPI.getAppConfig 回写 __authoritativeEdition 激活）
                    try { if (CONFIG.__authoritativeEdition) v = String(CONFIG.__authoritativeEdition); } catch(_)
{}
 // 时序竞态兜底（E1 偶发超时修复）：Permission.init 的
 // IPC 回调早于 index.html 内嵌 const CONFIG 声明落地时，CONFIG 处于 TDZ，
 // 权威 edition 改暂存于 Permission 实例（permission.js 先于内嵌脚本加载，
 // 实例必然存在）。插槽为空时由此兜底，userData 权威值不再被 asar 出厂
 // 默认(cloud_personal)经同步 XHR 反向掩盖 → 机构版用户管理按钮不再消失。
                    try { if (!v && global.Permission && global.Permission._authoritativeEdition) v =
String(global.Permission._authoritativeEdition); } catch(_) {}
 // 2. 否则回落到存储槽
                    if (!v) v = String(_slot || '');
 // 铁闸 3 getter 兜底：绕过 setter 直写 __authoritativeEdition 的值也要归一化
                    return __normalizeEdition(v);
                },
                set: function (v) {
 // 铁闸 3 setter 归一化：任何写 CONFIG.edition 的代码都会自动写规范 key
                    var val = __normalizeEdition(String(v || '').trim());
                    _slot = val;
 // 运行时写同步权威插槽：Permission.init 已把 userData 权威 edition
 // 写入 __authoritativeEdition（getter 最优先读取）。若 setter 不同步该插槽，
 // 登录版本切换（auth-core CONFIG.edition=targetEd）/ 启动缓存恢复等合法运行时
 // 写入会被 init 时的快照永久掩盖（"写入无效"回退）。故每次 setter 写入同时
 // 刷新权威插槽 = 最新写入者胜出，与 getter 优先级闭环自洽。
                    try { CONFIG.__authoritativeEdition = val; } catch(_) {}
 // 三写同步，保持向后兼容
                    try { global.EDITION = val; } catch(_) {}
```

```
                    try { if (global.Permission) global.Permission._edition = val; } catch(_) {}
                    try { if (global.Permission && typeof global.Permission.setEdition === 'function') {} /*
setEdition 里已会写三处，避免循环 */ } catch(_) {}
 // 写成功后，再触发一次按钮对齐
                    try { if (typeof global.__applyUserButtons === 'function') global.__applyUserButtons(); }
catch(_) {}
                    return true;
                }
            });
            CONFIG.__editionLocked = true;
 // 若 Permission 已存在，同步其 _edition 到 CONFIG 现在的值
            try {
                var cur = CONFIG.edition;
                if (cur && global.EDITION !== cur) global.EDITION = cur;
                if (cur && global.Permission && global.Permission._edition !== cur) global.Permission._edition =
cur;
            } catch(_) {}
            return true;
        } catch (e) {
 // TypeError: Cannot redefine property（理论上不会发生，因为 CONFIG 是 object literal 默认 configurable）
            try { if (console && console.warn) console.warn('[edition-lock] 锁失败，退化为轮询同步', e.message ||
e); } catch(_) {}
 // 退化为 2s 轮询兜底同步（10 次后停止）
            var ticks = 0;
            var timer = setInterval(function () {
                try {
                    if (++ticks > 10) { clearInterval(timer); return; }
                    var ce = ''; try { ce = String(CONFIG.edition || ''); } catch(_) {}
                    if (ce) {
                        try { if (global.EDITION !== ce) global.EDITION = ce; } catch(_) {}
                        try { if (global.Permission && Permission._edition !== ce) global.Permission._edition = ce;
} catch(_) {}
                    }
                } catch(_) {}
            }, 2000);
            return false;
        }
    }
    function activateOnReady() {
        if (document.readyState === 'loading') {
            document.addEventListener('DOMContentLoaded', function () { tryInstallLock(); });
        } else {
            tryInstallLock();
        }
 // load 再试一次（极端情况下 CONFIG 在 DOMContentLoaded 后才定义）
        window.addEventListener('load', function () { if (!CONFIG || !CONFIG.__editionLocked) tryInstallLock(); });
    }
 // 立即尝试；若 CONFIG 还未定义，会在 DOMContentLoaded 再试
    if (typeof CONFIG !== 'undefined') {
/* ========== 源文件：electron/prescription-counter.js（试用计数 AES-256-CBC + HMAC-SHA256） ========== */
// ============================================================================
// v3 新版：AES-256-CBC + HMAC 签名
// ============================================================================
function getMachineId() {
    try {
```

```
        return licenseManager.getMachineId();
    } catch (e) {
        return null;
    }
}
function deriveKey(machineId, salt) {
    return crypto.createHash('sha256').update(machineId + ':' + COUNT_KEY + ':' + salt).digest();
}
function aesEncrypt(text, machineId) {
    const key = deriveKey(machineId, 'count:v2');
    const hmacKey = deriveKey(machineId, 'hmac:v2');
    const iv = crypto.randomBytes(16);
    const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
    const encrypted = Buffer.concat([cipher.update(text, 'utf8'), cipher.final()]);
    const ivHex = iv.toString('hex');
    const encHex = encrypted.toString('hex');
    const hmac = crypto.createHmac('sha256', hmacKey).update(ivHex + ':' + encHex).digest('hex');
    return ENC2_PREFIX + ivHex + ':' + encHex + ':' + hmac;
}
function aesDecrypt(data, machineId) {
    try {
        const parts = data.split(':');
        if (parts.length !== 4 || parts[0] !== 'ENC2') return null;
        const ivHex = parts[1];
        const encHex = parts[2];
        const hmac = parts[3];
        const key = deriveKey(machineId, 'count:v2');
        const hmacKey = deriveKey(machineId, 'hmac:v2');
        const expectedHmac = crypto.createHmac('sha256', hmacKey).update(ivHex + ':' + encHex).digest('hex');
 // 常量时间比较防时序攻击
        if (hmac.length !== expectedHmac.length) return null;
        let diff = 0;
        for (let i = 0; i < hmac.length; i++) {
            diff |= hmac.charCodeAt(i) ^ expectedHmac.charCodeAt(i);
        }
        if (diff !== 0) return null;  // HMAC 校验失败
        const iv = Buffer.from(ivHex, 'hex');
        const encrypted = Buffer.from(encHex, 'hex');
        const decipher = crypto.createDecipheriv('aes-256-cbc', key, iv);
        const decrypted = Buffer.concat([decipher.update(encrypted), decipher.final()]);
        return decrypted.toString('utf8');
    } catch (e) {
        return null;
    }
}
// ============================================================================
// 路径与文件读写
// ============================================================================
function getCountPath() {
    return path.join(app.getPath('userData'), COUNT_FILE);
}
function readCounts() {
    try {
        const countPath = getCountPath();
        if (!fs.existsSync(countPath)) return {};
```

```
        const content = fs.readFileSync(countPath, 'utf8').trim();
 // v3：优先尝试 AES 解密
        const machineId = getMachineId();
        if (machineId && content.startsWith(ENC2_PREFIX)) {
            const json = aesDecrypt(content, machineId);
            if (json) {
                const data = JSON.parse(json);
                return (data && typeof data === 'object') ? data : {};
            }
 // AES 解密失败（可能是 machineId 变化或文件损坏），返回空
            console.error('[PrescriptionCounter] AES 解密失败，计数文件可能被篡改');
            return {};
        }
 // 迁移：旧 XOR 格式 → 新 AES 格式
        const oldJson = xorDecrypt(content, COUNT_KEY);
        if (oldJson) {
            try {
                const data = JSON.parse(oldJson);
                if (data && typeof data === 'object') {
                    console.log('[PrescriptionCounter] 检测到旧 XOR 格式，自动迁移到 AES');
 // 用新格式重新保存
                    writeCounts(data);
                    return data;
                }
            } catch (e) { /* JSON 解析失败，继续 */ }
        }
        return {};
    } catch (e) {
        console.error('[PrescriptionCounter] 读取计数失败:', e.message);
        return {};
    }
}
function writeCounts(counts) {
    try {
        const countPath = getCountPath();
        const json = JSON.stringify(counts);
 // v3：使用 AES 加密
        const machineId = getMachineId();
        if (machineId) {
            const encrypted = aesEncrypt(json, machineId);
            fs.writeFileSync(countPath, encrypted, 'utf8');
        } else {
 // 极端降级：machineId 不可用时使用 XOR（向后兼容）
            const encrypted = xorEncrypt(json, COUNT_KEY);
            fs.writeFileSync(countPath, encrypted, 'utf8');
        }
    } catch (e) {
        console.error('[PrescriptionCounter] 写入计数失败:', e.message);
    }
}
/* ========== 源文件：button-manager.js（按钮单一写入源 / 版本按钮统一装配） ========== */
    function __applyUserButtons(user, edition) {
        var p = __computePermissions(user, edition);
        var umb = document.getElementById('userManageBtn');
        var cpb = document.getElementById('changePwdBtn');
```

```
        var cpr = document.getElementById('clinicPrescriptionBtn');
 // ——— Arch 2.26 可观测水印：把架构版本+修复 ID 写进 title，用户悬停按钮就能判断是否新包生效
 //  （不改变视觉文字，只改 tooltip，避免 UI 基线 SHA256 漂移）
 // 2.25 = edition 别名归一化（institution/standard/中文→规范 key）+ Single-Writer 断言
 // 2.26 = 登录页三元组双挂载点 + 两行紧凑布局 + 窗口加高 30px + 全局推广到 db-offline
        var _archWatermark = 'Arch 2.26 | editionNormalize | instAdminAssert | roleDowngradeGuard';
        if (umb) { umb.style.display = p.canManage ? 'block' : 'none'; umb.style.visibility = p.canManage ?
'visible' : 'hidden'; umb.setAttribute('title', (umb.getAttribute('title') ? umb.getAttribute('title').split(' ||
  ')[0] + ' || ' : '') + _archWatermark); }
        if (cpb) { cpb.style.display = p.canChangePwd ? 'block' : 'none'; cpb.style.visibility = p.canChangePwd ?
'visible' : 'hidden'; cpb.setAttribute('title', (cpb.getAttribute('title') ? cpb.getAttribute('title').split(' ||
  ')[0] + ' || ' : '') + _archWatermark); }
        if (cpr) { cpr.style.display = p.canViewAll ? 'block' : 'none'; cpr.style.visibility = p.canViewAll ?
'visible' : 'hidden'; cpr.setAttribute('title', (cpr.getAttribute('title') ? cpr.getAttribute('title').split(' ||
  ')[0] + ' || ' : '') + _archWatermark); }
 // 移动端按钮 btn2（【用户管理】→ 改密码在移动端是改密图标，无移动端时无此按钮）
        try {
            var btn2 = document.getElementById('mobileBtn2');
            if (btn2) {
                if (p.isPersonal) {
                    btn2.setAttribute('data-action', 'changePassword');
                    var icon = btn2.querySelector('.icon'); if (icon) icon.textContent = '🔗';
                    var label = btn2.querySelector('.label'); if (label) label.textContent = '改密';
                } else if (p.isInst) {
                    btn2.setAttribute('data-action', 'userManage');
                    var _icon2 = btn2.querySelector('.icon'); if (_icon2) _icon2.textContent = '👥';
                    var _label2 = btn2.querySelector('.label'); if (_label2) _label2.textContent = '用户';
                }
                btn2.style.display = '';
            }
        } catch(_) {}
        return p;
    }
 // ———————————————— Setup 权威模式的 edition 纠正（只改状态不碰 DOM，DOM 由 __applyUserButtons 统一写）
————————————————
    function __enforceStandardEditionInner(reason) {
 // 先收集所有不依赖 window.APP_MODE 的云端/机构线索（APP_MODE 在 cloud_desktop/index.html L680+
才定义，晚于本脚本）
        var tag = (typeof getEditionTag === 'function') ? String(getEditionTag()) : '';
        var globalProduct = (typeof window.PRODUCT_NAME !== 'undefined') ? String(window.PRODUCT_NAME) : '';
        var configProduct = ''; try { if (typeof CONFIG !== 'undefined') configProduct = String(CONFIG.productName
|| ''); } catch(_) {}
        var configEdition = ''; try { if (typeof CONFIG !== 'undefined') configEdition = String(CONFIG.edition ||
''); } catch(_) {}
        var globalEdition = String(global.EDITION || '');
        var domMarker = !!(document.getElementById('_force_standard_edition_marker_'));
        var permEdition = ''; try { if (global.Permission && Permission._edition) permEdition =
String(Permission._edition); } catch(_) {}
        var cloudApiBaseExists = (typeof CLOUD_API_BASE !== 'undefined');
        var urlHasCloud = false;
        try {
            var _loc = String(document.location && document.location.href ? document.location.href : '');
            var _path = String(document.location && document.location.pathname ? document.location.pathname : '');
            urlHasCloud = (_loc.indexOf('db-yunduan') >= 0 || _path.indexOf('db-yunduan') >= 0 ||
_loc.indexOf('/cloud_') >= 0 || _path.indexOf('/cloud_') >= 0);
```

```
        } catch(_) {}
        var anyCloudHint = __isCloudEditionStr(configEdition) || __isCloudEditionStr(globalEdition) ||
__isCloudEditionStr(permEdition) || cloudApiBaseExists || urlHasCloud;
 // ——— ① isDesktopLocal 重写：有任何云端/机构线索立即=false，不再等 APP_MODE ———
        var isDesktopLocal = false;
        try {
            var isElectronNow = false;
            try { if (typeof IS_ELECTRON !== 'undefined' && IS_ELECTRON) isElectronNow = true; } catch(_) {}
            try { if (global.electronAPI && global.electronAPI.isElectron) isElectronNow = true; } catch(_) {}
            if (isElectronNow) {
 // Electron 环境下判离线权威：必须显式 APP_MODE === 'offline'，或无任何云端线索
                var appModeOffline = false;
                try { appModeOffline = (String(global.APP_MODE || '') === 'offline'); } catch(_) {}
                var appModeCloud = false;
                try { appModeCloud = (String(global.APP_MODE || '') === 'cloud'); } catch(_) {}
                if (appModeCloud || anyCloudHint) {
 // 有 APP_MODE=cloud 或任一条云端线索 → 明确是云端 Electron，不启用 Setup 离线权威
                    isDesktopLocal = false;
                } else if (appModeOffline) {
                    isDesktopLocal = true;
                } else if (globalProduct === '惠康中医-本地' || configProduct === '惠康中医-本地') {
                    isDesktopLocal = true;
                } else if (__isInstStr(configEdition) || __isInstStr(globalEdition) || __isInstStr(permEdition)) {
 // 任一处 edition 明确机构版 → 永远不启用权威离线强制（会误伤身份）
                    isDesktopLocal = false;
                } else if (tag.indexOf('云端') >= 0) {
                    isDesktopLocal = false;
                } else {
 // 最保守：APP_MODE 未定义又无任何云端线索 + 是 Electron → 暂保留 true（老离线项目）
                    isDesktopLocal = true;
                }
            }
        } catch(_) { isDesktopLocal = false; }
 // ——— 云端宽判（保留，给 isStandardSoft 的 softMarker 用）———
        var isCloudProduct = false;
        try {
            if (String(global.APP_MODE || '') === 'cloud') isCloudProduct = true;
            if (String(globalProduct) === '惠康中医-云端' || String(configProduct) === '惠康中医-云端')
isCloudProduct = true;
            if (anyCloudHint) isCloudProduct = true;  // 汇总已有所有云端线索，避免重复判断
        } catch(_) {}
        var editionCloudHint = __isCloudEditionStr(configEdition) || __isCloudEditionStr(globalEdition) ||
__isCloudEditionStr(permEdition);
        var softMarker = !isCloudProduct && !editionCloudHint && domMarker;
 // ——— ② institutionExempt：增加 localStorage 预登录管理员豁免（防止登录前把 role 打成 user）———
        var _cfgIsInst = __isInstStr(configEdition);
        var _winIsInst = __isInstStr(globalEdition);
        var _permIsInst = __isInstStr(permEdition);
        var _storedRole = '';
        try {
            var _usRaw = localStorage.getItem('currentUser');
            if (_usRaw) { try { var _usObj = JSON.parse(_usRaw); _storedRole = String(_usObj.role ||
'').toLowerCase().trim(); } catch(_) {} }
        } catch(_) {}
        var _memoryRole = '';
```

```
        try { if (typeof currentUser !== 'undefined' && currentUser) _memoryRole = String(currentUser.role ||
'').toLowerCase().trim(); } catch(_) {}
        var _roleSaysAdmin = (_storedRole === 'admin' || _storedRole === 'clinic_admin' || _memoryRole === 'admin'
|| _memoryRole === 'clinic_admin');
        var institutionExempt = _cfgIsInst || _winIsInst || _permIsInst || _roleSaysAdmin;
 // ——— isStandardSoft：保留，但机构版豁免已含 role 线索 → 不会误触发标准版 ———
        var isStandardSoft = false
            || (tag.indexOf('标准版') >= 0 && !__isInstStr(configEdition) && !__isInstStr(globalEdition))
            || (globalProduct === '惠康中医-本地')
            || (configProduct === '惠康中医-本地')
            || (['personal', 'offline_personal'].indexOf(configEdition) >= 0) && !__isInstStr(globalEdition) &&
!_roleSaysAdmin)
            || ((globalEdition === 'personal' || globalEdition === 'offline_personal') &&
!__isInstStr(configEdition) && !_roleSaysAdmin)
            || softMarker;
        var mustEnforce = isDesktopLocal || isStandardSoft;
        if (!institutionExempt && mustEnforce) {
 // A) edition 强制 personal（通过归一化锁 setter 自动三写同步）
            try {
                if (typeof CONFIG !== 'undefined' && CONFIG) CONFIG.edition = 'personal';
                else global.EDITION = 'personal';
            } catch(_) { try { global.EDITION = 'personal'; } catch(_) {} }
            try { if (global.Permission && typeof Permission.setEdition === 'function')
Permission.setEdition('personal'); } catch(_) {}
            try { if (global.Permission) Permission._edition = 'personal'; } catch(_) {}
 // ——— ③ role 强改写死保护：admin/clinic_admin 身份永不打 user ———
            try {
                var _mustNotDowngrade = _roleSaysAdmin;  // 存储或内存里有 admin/clinic_admin
 // 额外：明确机构版 edition 的任何一处也不允许降权
                if (!_mustNotDowngrade && (_cfgIsInst || _winIsInst || _permIsInst)) _mustNotDowngrade = true;
                if (!_mustNotDowngrade) {
 // 只有"edition 全是个人版线索 + role 不是 admin/clinic_admin"时才允许降级
                    if (typeof currentUser !== 'undefined' && currentUser) currentUser.role = 'user';
                    var _us3 = localStorage.getItem('currentUser');
                    if (_us3) { try { var _u3 = JSON.parse(_us3); _u3.role = 'user';
localStorage.setItem('currentUser', JSON.stringify(_u3)); } catch(_) {} }
                }
            } catch(_) {}
        }
 // 所有出口（豁免 OR 强制执行完毕）最后统一由 Single-Writer 对齐 DOM，永远一致
        try { return __applyUserButtons(); } catch(_) {}
    }
 // ——————————— 运行时替换所有旧调用方 ———————————
    function __patchOldCallers() {
 // 1) 覆盖全局 enforceStandardEditionButtons → 统一走权威模式 + 单一写入
        global.enforceStandardEditionButtons = function (reason) {
            try { return __enforceStandardEditionInner(reason); } catch(e) {
                try { if (typeof console !== 'undefined' && console.warn) console.warn('[btn-mgr] enforce
异常（已吞）:', e.message || e); } catch(_) {}
 // 异常兜底最后也走 Single-Writer 保证一致
                try { return __applyUserButtons(); } catch(_) {}
            }
        };
 // 2) 覆盖 Permission.applyRuntimePermissions → 机构版/个人版都只委托 Single-Writer
        if (global.Permission && Permission.applyRuntimePermissions) {
```

```
            var _orig = Permission.applyRuntimePermissions;
            Permission.applyRuntimePermissions = function () {
                try { _orig.call(Permission); } catch(_) {}
 // 覆盖旧函数里的不对称隐藏：让 Single-Writer 以最新权威状态重算并落 DOM
                try { __applyUserButtons(); } catch(_) {}
 // 同步入口屏蔽（非云端版）——旧逻辑，保留（非三件套按钮，不属于 Single-Writer 范围）
                try {
                    var isCloudNow = false;
                    try {
                        if (typeof CONFIG !== 'undefined' && CONFIG) {
                            var ed2 = String(CONFIG.edition || '');
                            isCloudNow = __isCloudEditionStr(ed2) || __isInstStr(ed2) /*
机构版默认走云端同步权限显示 */ ||
                                ['cloud','cloud_personal','cloud_clinic','institution','clinic_custom'].indexOf(ed2)
 >= 0;
                        }
                    } catch(_) {}
                    if (!isCloudNow) {
                        var nds = document.querySelectorAll('[onclick*=\"sync\"], #syncBtn, #cloudSyncBtn,
#syncStatus');
                        if (nds && nds.forEach) nds.forEach(function(el){ try { el.style.display = 'none'; }
catch(_){} });
                    }
                } catch(_) {}
            };
        }
 // 3) Wrapper updateUserDisplay：旧函数还是会做业务逻辑（用户名显示/基础设置等），但三件套按钮由 Single-Writer
最终覆盖
        if (typeof global.updateUserDisplay === 'function') {
            var _origUpdateUser = global.updateUserDisplay;
            global.updateUserDisplay = function () {
                try { _origUpdateUser.apply(global, arguments); } catch(e) {
                    try { if (console && console.warn) console.warn('[btn-mgr] updateUserDisplay 异常:', e.message
|| e); } catch(_) {}
                }
 // 旧 updateUserDisplay 内部也给三件套按钮赋值（多写源），这里用 Single-Writer 最终覆盖掉
                try { __applyUserButtons(); } catch(_) {}
            };
        }
    }
 // ———————————— 公共 API 暴露 ————————————
    global.__applyUserButtons = __applyUserButtons;
    global.__computePermissions = __computePermissions;
    global.__patchOldCallers = __patchOldCallers;
    global.__INSTITUTIONAL_EDITIONS = INSTITUTIONAL_EDITIONS;
    global.__PERSONAL_EDITIONS = PERSONAL_EDITIONS;
 // 立即打补丁（DOMContentLoaded 前执行也 OK，因为只是覆盖函数对象，不访问 DOM）
    if (document.readyState === 'loading') {
        document.addEventListener('DOMContentLoaded', function () { try { __patchOldCallers(); } catch(_) {} });
 // 补丁生效后再在 load 时跑一次对齐（此时 Permission.init 已就绪）
        window.addEventListener('load', function () { try { __applyUserButtons(); } catch(_) {} });
    } else {
        try { __patchOldCallers(); } catch(_) {}
    }
})(window);
```